

两级运载火箭发射轨迹建模与入轨优化

摘 要

两级运载火箭的精准入轨与推进剂优化是航天工程的核心课题。本文针对赤道发射、目标为 400 km 近地圆轨道的两级液体运载火箭，建立了考虑地球自转、旋转大气阻力、变质量与级间质量跳变的统一多阶段动力学模型，并按照“给定控制传播—有限维参数反演—推力方向最优控制—方向与节流联合控制”的层级逐步开放决策自由度，使四问共享同一物理模型、终端流形与验证标准。

针对问题一，建立了地心惯性系下的平面点质量动力学模型，以赤道东射初速 $v_0 = \omega_E R_E$ 刻画地球自转优势，以相对大气速度计算阻力，以真空比冲确定质量流量，并采用分段自适应数值积分附事件检测完成基准策略全程仿真。结果表明：二子级燃尽时高度 **433.53 km**、惯性速度 **8521.18 m/s**、飞行路径角 -0.69° ，对应近地点约 431 km、远地点约 4723 km 的椭圆轨道而非目标圆轨道。该结果直接说明基准策略燃料打过头，必须提前关机并优化控制律。

针对问题二，将入轨条件写成半径、径向速度与顺行切向圆轨道速度三个等式，构建滑行时间、俯仰角角速率与二级燃烧时间组成的三维非线性打靶模型，通过 196 组网格初值加 fsolve 求解。得到可行入轨解：滑行时间 **103.63 s**，角速率 $-0.0474^\circ/\text{s}$ ，剩余推进剂约 **4015 kg**，入轨残差达 10^{-9} 量级。该问是可行解反演，无目标函数，属于参数搜索而非优化。

针对问题三，将俯仰角由单一速率参数提升为可自由选取的时间函数，建立以推进剂消耗为目标、以状态轨迹与控制函数为决策变量的多阶段最优控制模型，采用 Hermite-Simpson 直接配点将微分方程约束、阶段连接与终端流形同时转写为稀疏非线性规划，用 CasADi 自动微分与内点法求解。由于恒定满推力且比冲恒定，最省燃料等价于最短燃烧时间。20/40 区间网格求得滑行时间 **4.3767 s**、二级燃烧时间 **394.353 s**、推进剂消耗 **57446.93 kg**，与 10/20 网格仅差 0.012 kg；所得最优控制经独立高精度积分复验，终端高度误差小于 1 m。与问题二相比耗药减少约 538 kg，且最优滑行时间由 103.63 s 缩短至 4.38 s，说明旧的低维参数化限制了可行轨迹族，连续推力方向模型释放了新的控制自由度。

针对问题四，在问题三基础上引入节流比 $\sigma(t) \in [0.6, 1]$ ，并正确将推进剂容量约束改写为累计推力积分约束，以同一配点转录求解。从满推力与 80% 推力两类初值均收敛到同一近似满推力候选解（ σ 节点位于 0.99998~1.00000），滑行时间 4.3764 s，耗药 57446.93 kg，与问题三一致。本文结合固定比冲下的重力损失机理与文献结论指出，在无最大动压、过载、热流等路径约束的本题设定下节流不产生收益；进一步讨论了加入路径约束后节流可能出现的非平凡切换结构。鉴于问题非凸，第三、四问的结论均表述为经网格一致性与独立复验支持的局部最优候选解，不作全局最优性保证。

关键词： 运载火箭 多阶段最优控制 直接配点 入轨打靶 推力节流

一、 问题重述

1.1 问题背景

近年来,随着深空探测与大型空间站建设需求的增加,重型运载火箭的精准入轨与推进剂优化成为航天工程的核心课题。火箭的发射过程受地球自转、重力场变化、大气阻力以及火箭自身质量不断减少等多因素协同影响。为将有效载荷以最低能量消耗送入预定轨道,必须对发射轨迹进行精准建模,并对关键飞行阶段的推力大小与方向角进行优化设计。

1.2 已知参数与任务

某两级液体运载火箭在赤道发射场发射,目标为将有效载荷送入高度 $H = 400 \text{ km}$ 的近地圆轨道。火箭基本参数:一子级起飞质量 $M_0 = 500\,000 \text{ kg}$, 结构质量 $m_{s1} = 40\,000 \text{ kg}$, 推进剂质量 $m_{p1} = 380\,000 \text{ kg}$, 真空比冲 $I_{sp1} = 300 \text{ s}$, 额定推力 $F_{\max 1} = 7.5 \times 10^6 \text{ N}$, 阻力参考面积 $S = 12.5 \text{ m}^2$, 阻力系数 $C_D = 0.3$; 二子级结构质量 $m_{s2} = 8\,000 \text{ kg}$, 推进剂质量 $m_{p2} = 62\,000 \text{ kg}$, 有效载荷 $m_{pl} = 10\,000 \text{ kg}$, 真空比冲 $I_{sp2} = 420 \text{ s}$, 额定推力 $F_{\max 2} = 6.0 \times 10^5 \text{ N}$; 一子级分离后二子级气动阻力忽略; 大气密度采用指数模型 $\rho(h) = 1.225 e^{-h/7200}$ 。

飞行过程分三个阶段: I 垂直起飞与程序转弯, 一子级工作, 首先垂直上升 10 s , 之后推力俯仰角以恒定速度 $0.4^\circ/\text{s}$ 线性减小直至一子级推进剂耗尽关机分离; II 无动力滑行, 二子级不点火, 依靠惯性滑行设定时间; III 入轨修正, 二子级点火, 通过调整推力大小与俯仰角使有效载荷达到入轨条件。

需要解决以下问题:

1. 建立动力学模型, 在基准控制策略下仿真全程, 给出高度、速度、飞行路径角与总质量曲线, 并计算二子级关机时刻的轨道高度、速度与飞行路径角。基准策略为: 一级额定推力; 转弯段俯仰角以 $0.4^\circ/\text{s}$ 线性减小; 二级全额推力且推力方向始终与速度方向一致; 滑行时间 60 s 。
2. 设定二子级推力俯仰角以恒定速率变化, 可提前关机, 关机时间由入轨条件决定; 调整滑行时间与俯仰角变化速率, 初始俯仰角与速度方向一致, 使卫星进入 400 km 近地圆轨道。
3. 设计滑行时间与二子级推力俯仰角的优化控制策略, 使二子级消耗燃料最省。
4. 若二子级发动机具备推力节流能力, 推力可在 $60\% \sim 100\%$ 额定推力间连续调节, 重新设计滑行时间与二子级推力大小、俯仰角的优化控制策略, 使燃料消耗最省。

二、 问题分析

2.1 问题一的分析

问题一要求建立物理机理模型。火箭在赤道平面内发射并进入赤道目标圆轨道，运动对称退化为平面点质量问题。关键建模要素有三：地球自转决定初速与入轨速度基准，指数大气决定阻力，阻力仰赖相对大气速度；质量流动决定推力方程与燃尽时刻。基准策略控制律确定后，问题一化为确定性的分阶段初值问题，采用高精度自适应积分并附带事件检测即可求解，重点在于物理量一致性与阶段衔接的严谨性。

2.2 问题二的分析

入轨条件为目标半径、零径向速度和顺行切向圆轨道速度三个等式；未知量取滑行时间、俯仰角速率和二级燃烧时间，形成三维封闭非线性打靶系统。该问只要求找到满足终端流形的参数组合，没有额外目标函数，因此所得结果是可行入轨解而非最优解。滑行段的状态预报精度会直接影响后续边值迭代质量 [5]。

2.3 问题三的分析

在问题二基础上将俯仰角由单一速率参数提升为时间函数，并同时优化滑行与关机时刻。恒定满推力、恒定比冲使推进剂消耗与二级燃烧时间严格成正比，故本问的模型本质是在圆轨道终端流形上寻找最短可行燃烧时间。采用直接配点把状态方程、阶段连接和控制边界同时转写为稀疏非线性规划，可避免单重打靶把全部灵敏度压缩在终点的缺点 [3]。

2.4 问题四的分析

在问题三基础上增加节流比控制量，推进剂消耗变为 $\int \sigma(t) dt$ 的函数。此时实际工作时间可以超过满推力燃尽时长，容量约束必须写成累计耗药量或末质量下界，不能继续限制 $t_3 - t_2 \leq t_{b2}$ 。通过与问题三共用的配点模型及低节流初值同伦，检验节流是否产生真实收益；路径约束对节流结构的影响参照相关研究讨论 [1]。

三、 模型假设

1. 火箭在赤道平面内运动，目标轨道为赤道平面内圆轨道，运动退化为平面问题；
2. 火箭视为质点，忽略姿态动力学与攻角产生升力，只考虑推力、重力、气动阻力；
3. 地球为均质球体，引力加速度 $\mathbf{g} = -\mu \mathbf{r}/|\mathbf{r}|^3$ ，其中 $\mu = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2$ ，地球半径取 $R_E = 6371 \text{ km}$ ；
4. 大气随地球同步旋转，阻力使用相对地球大气速度 $\mathbf{v}_{\text{rel}} = \mathbf{v} - \boldsymbol{\omega}_E \times \mathbf{r}$ ，大气密度按

指数模型计算；

5. 问题一至三的二级发动机采用额定推力，问题四允许在 60%–100% 范围内节流；质量流量均由真空比冲确定；
6. 一子级分离后，二子级与有效载荷的气动阻力忽略不计；
7. 一子级分离为瞬时事件，分离时质量发生跳变，速度位置连续；
8. 目标圆轨道为重力场内的惯性圆轨道，入轨判据均以惯性系速度为准；
9. 飞行动力学中推力方向始终由俯仰角控制，无侧向控制量。

四、 符号说明

符号	含义	符号	含义
x, y, v_x, v_y	平面惯性位置/速度分量	μ	地球引力常数
m	当前总质量	R_E	地球半径
r	地心距 $ r = \sqrt{x^2 + y^2}$	ω_E	地球自转角速度
h	高度 $h = r - R_E$	ρ	大气密度
T	推力大小	S, C_D	阻力参考面积/系数
ϕ	推力俯仰角，与当地水平面夹角	I_{sp}	真空比冲
γ	飞行路径角，速度与水平面夹角	g_0	海平面重力加速度
t_c	无动力滑行时间	k	二子级俯仰角变化率
t_1, t_2, t_3	一级关机/二级点火/二级关机时刻	σ	推力节流比
$\hat{r}, \hat{t}$	当地径向/东向单位向量	v_{rel}	相对大气速度

五、 问题一：动力学模型与基准控制策略仿真

5.1 坐标系与状态量

描述火箭运动的第一步是选择坐标系。火箭在赤道发射，目标为赤道平面内的圆轨道。考察发射全程的外力：重力始终指向地心，属于径向力；推力方向由俯仰角控制，本文策略中推力始终在赤道平面内；大气阻力方向与相对速度相反，只要相对速度位于赤道平面内阻力就不出该平面。初始位置在赤道平面内、初速沿赤道平面的东向，因此全部外力与初始条件均不产生垂直于赤道平面的分量，运动被约束在赤道平面内，即平

面问题。

在赤道平面内采用惯性系分量描述。取发射时刻发射点所在经线为 x 轴正方向，向东为 y 轴正方向， z 轴沿地球自转轴，则 z 方向分量恒为零。记火箭地心位置向量为 $\mathbf{r} = (x, y)$ ，速度向量为 $\mathbf{v} = (v_x, v_y)$ ，当前质量为 m ，则状态向量

$$\mathbf{x} = [x, y, v_x, v_y, m]^T. \quad (1)$$

火箭在发射台上随地球自转，发射点位于赤道，地心距为 R_E ，自转速度方向沿东向。由于 x 轴选取的是发射点所在经线，发射点即位于 x 轴上，故位置初值为 $(R_E, 0)$ ；自转线速度沿 y 轴方向，大小为 $\omega_E R_E$ 。因此初值

$$x(0) = R_E, \quad y(0) = 0, \quad v_x(0) = 0, \quad v_y(0) = \omega_E R_E, \quad m(0) = M_0. \quad (2)$$

该初值蕴含赤道东射获得的约 465 m/s 惯性切向速度，是地球自转在发射阶段的体现。这一初速度对入轨速度基准的影响将在 §5.3 中结合入轨条件说明。

5.2 受力分析

火箭在飞行中受到三类外力的作用，如图 1a 所示。

5.2.1 推力

发动机喷流形成推力，大小由发动机节流状态决定：问题一至三中二子级恒以额定满推力工作，问题四允许在 60%–100% 额定推力间连续调节（此时推力 $T = \sigma(t)F_{\max 2}$ ， σ 为节流比）。推力方向由喷管指向决定，本问题以俯仰角作为控制量，俯仰角定义为推力矢量与当地水平面的夹角。为把推力表达为惯性系，需要建立局部基向量。设当前地心位置向量为 $\mathbf{r}$ ，则当地径向单位向量

$$\hat{\mathbf{r}} = \frac{\mathbf{r}}{|\mathbf{r}|} = \left(\frac{x}{r}, \frac{y}{r} \right), \quad r = |\mathbf{r}| = \sqrt{x^2 + y^2}. \quad (3)$$

东向单位向量 $\hat{\mathbf{t}}$ 由径向单位向量绕 z 轴旋转 90° 得到，

$$\hat{\mathbf{t}} = \left(-\frac{y}{r}, \frac{x}{r} \right). \quad (4)$$

$\hat{\mathbf{r}}$ 与 $\hat{\mathbf{t}}$ 构成当地正交基，其中 $\hat{\mathbf{r}}$ 沿地心向外， $\hat{\mathbf{t}}$ 沿当地东向。推力矢量与 $\hat{\mathbf{t}}$ 的夹角为俯仰角 ϕ ，故推力方向单位向量

$$\hat{\mathbf{u}} = \cos \phi \hat{\mathbf{t}} + \sin \phi \hat{\mathbf{r}}. \quad (5)$$

$\phi = 90^\circ$ 时推力沿当地垂直方向，即垂直起飞姿态； $\phi = 0$ 时推力沿水平方向。将推力大小记为 T ，则推力向量 $\mathbf{T} = T\hat{\mathbf{u}}$ ，在惯性系下的分量为

$$T_x = T\hat{u}_x, \quad T_y = T\hat{u}_y. \quad (6)$$

5.2.2 重力

地球视为均质球体，引力场为中心引力场。引力加速度大小随地心距平方反比衰减，方向沿径向指向地心，即

$$\mathbf{g} = -\frac{\mu}{r^3}\mathbf{r}, \quad (7)$$

其中 $\mu = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2$ 为地球引力常数， r 为地心距。式 (7) 的直角坐标分量为

$$g_x = -\frac{\mu x}{r^3}, \quad g_y = -\frac{\mu y}{r^3}. \quad (8)$$

该式说明重力加速度只在近地高度范围内近似常数 g_0 ，随高度上升而减小，本题目标轨道对应重力加速度约 8.7 m/s^2 ，忽略重力随高度变化会引入不可接受的误差。

5.2.3 气动阻力

大气随地球同步旋转，阻力与火箭相对大气的运动有关。惯性系速度 $\mathbf{v}$ 与随地球旋转的大气速度之差为相对大气速度

$$\mathbf{v}_{\text{rel}} = \mathbf{v} - \boldsymbol{\omega}_E \times \mathbf{r}. \quad (9)$$

在平面问题中， $\boldsymbol{\omega}_E \times \mathbf{r} = \omega_E(-y, x)$ ，故分量形式

$$v_{\text{rel},x} = v_x + \omega_E y, \quad v_{\text{rel},y} = v_y - \omega_E x. \quad (10)$$

阻力方向与相对速度方向相反，大小由动压、参考面积与阻力系数决定。动压为 $\frac{1}{2}\rho(h)|\mathbf{v}_{\text{rel}}|^2$ ，其中大气密度按指数模型

$$\rho(h) = \rho_0 e^{-h/h_0}, \quad \rho_0 = 1.225 \text{ kg/m}^3, \quad h_0 = 7200 \text{ m}. \quad (11)$$

阻力向量为

$$\mathbf{D} = -\frac{1}{2}\rho(h)SC_D|\mathbf{v}_{\text{rel}}|\mathbf{v}_{\text{rel}}, \quad (12)$$

阻力在惯性系下的分量为

$$D_x = -\frac{1}{2}\rho(h)SC_D|\mathbf{v}_{\text{rel}}|v_{\text{rel},x}, \quad D_y = -\frac{1}{2}\rho(h)SC_D|\mathbf{v}_{\text{rel}}|v_{\text{rel},y}. \quad (13)$$

5.2.4 质量流出

发动机工作时以恒定质量流量排出推进剂。真空比冲 I_{sp} 定义为单位质量推进剂产生的冲量, $I_{sp} = T/(\dot{m}g_0)$, 故质量流量

$$\dot{m} = -\frac{T}{I_{sp} g_0}. \quad (14)$$

质量流量为负表示火箭质量随时间减少。发动机关机或滑行时 $T = 0$, 质量保持不变。

5.3 动力学方程

将各外力表达式代入牛顿第二定律 $\mathbf{F} = m\mathbf{a}$, 即得平面动力学方程。式 (15) 由式 (14) 代入 $T = 0$, 式 (17)–(18) 右端三项依次为重力、推力与阻力加速度, 分别来自式 (8)、(6) 与 (13),

$$\dot{m} = -\frac{T}{I_{sp} g_0}, \quad (15)$$

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad (16)$$

$$\dot{v}_x = -\frac{\mu x}{r^3} + \frac{T}{m} \hat{u}_x + \frac{D_x}{m}, \quad (17)$$

$$\dot{v}_y = -\frac{\mu y}{r^3} + \frac{T}{m} \hat{u}_y + \frac{D_y}{m}. \quad (18)$$

其中式 (15) 描述推进剂消耗, 质量流量由真空比冲与推力确定; 式 (16) 为速度与位置的运动学关系; 式 (17)–(18) 为惯性系下的动力学方程, 右端三项依次为重力、推力与阻力加速度, 量纲均为加速度, 可直接用于积分。

飞行路径角 γ 定义为速度向量与当地水平面的夹角, 由惯性速度的径向与切向分量确定,

$$\gamma = \text{atan2}(\mathbf{v} \cdot \hat{\mathbf{r}}, \mathbf{v} \cdot \hat{\mathbf{t}}). \quad (19)$$

图 1a 给出坐标系与受力示意, 图 2 给出飞行剖面。

5.4 分阶段控制策略

按题面设定, 全程划分为四个时间区间, 如表 1 所示。

一子级燃尽时刻 $t_1 = m_{p1}/\dot{m}_1 = 149.06$ s, 二子级燃尽时长 $t_{b2} = m_{p2}/\dot{m}_2 = 425.61$ s。滑行期 $t_c = 60$ s, 故 $t_2 = t_1 + t_c$ 。

5.5 多阶段状态转移与数值验证

全程不是一个光滑的单段常微分方程, 而是由动力段、滑行段和分离跳变组成的混合动力系统。记一级动力、滑行和二级动力的状态转移分别为 Φ_1 、 Φ_c 和 Φ_2 , 分离质量

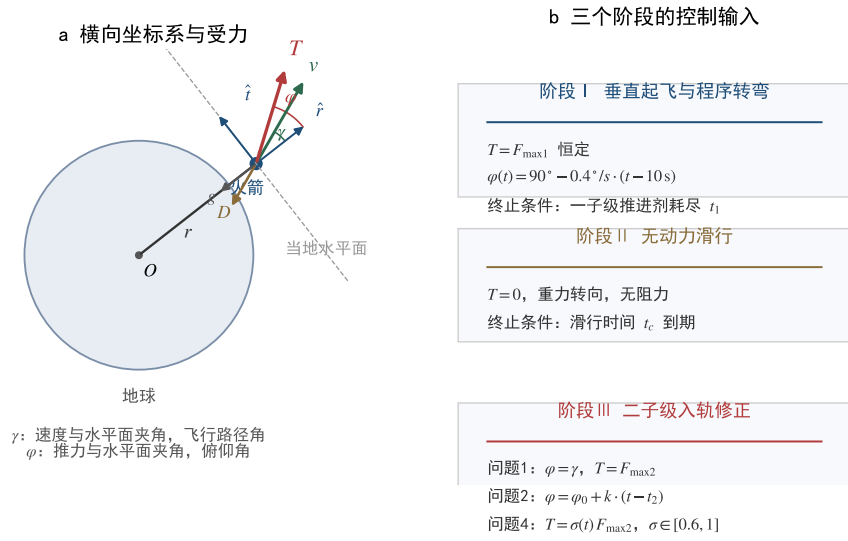


图 1 受力分析

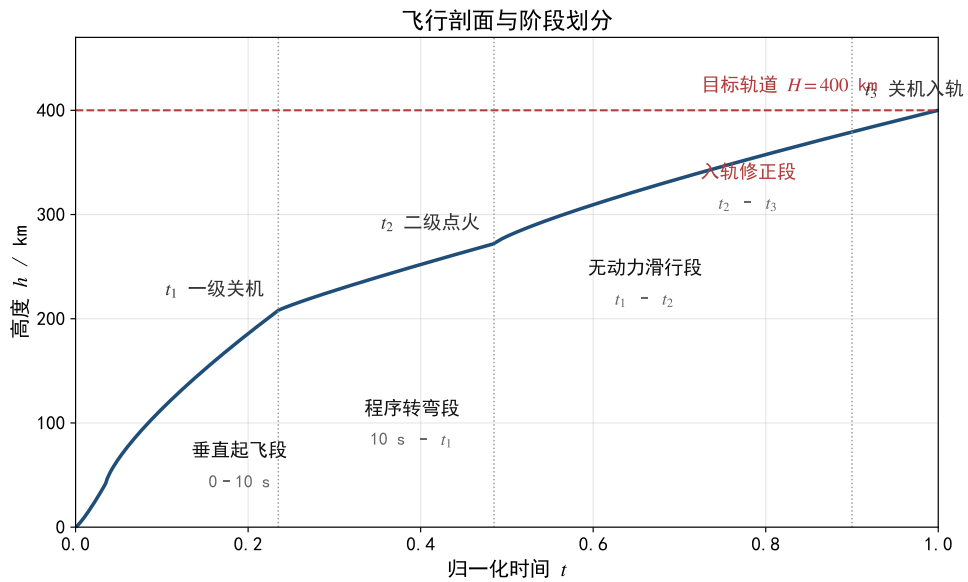


图 2 飞行剖面

表 1 基准控制策略

阶段	时间区间	推力 T	俯仰角 ϕ
垂直起飞	$[0, 10]\text{ s}$	$F_{\max 1}$	90°
程序转弯	$[10, t_1]$	$F_{\max 1}$	$90^\circ - 0.4^\circ/\text{s} \cdot (t - 10)$
一子级分离	t_1	—	$m \leftarrow m - m_{s1}$
无动力滑行	$[t_1, t_2]$	0	—
二子级动力	$[t_2, t_3]$	$F_{\max 2}$	与惯性速度方向一致, 攻角为零

跳变为 $\mathcal{J}$ ，则关机状态可写成

$$\mathbf{x}(t_3) = \Phi_2(t_3, t_2; \mathcal{J} [\Phi_c(t_2, t_1; \Phi_1(t_1, 0; \mathbf{x}_0))]) . \quad (20)$$

这一表达式明确了位置、速度在阶段连接处连续，而一级分离只改变质量。问题一控制律给定后，式 (20) 是确定的初值问题；采用自适应高阶 Runge-Kutta 方法 (DOP853) 逐段传播，并以推进剂耗尽事件截断动力段。相对容差由 10^{-7} 缩小到 10^{-11} 时，问题一关机高度变化小于 0.04 m、速度变化小于 10^{-3} m/s，说明积分误差远小于模型误差。问题三、四的控制由直接配点求得后，再用该分段传播器独立重积分，避免把优化器内部缺陷误当成真实终端精度。

5.6 结果与分析

全程仿真结果如图 3 与表 2 所示。

问题一：基准策略全程曲线

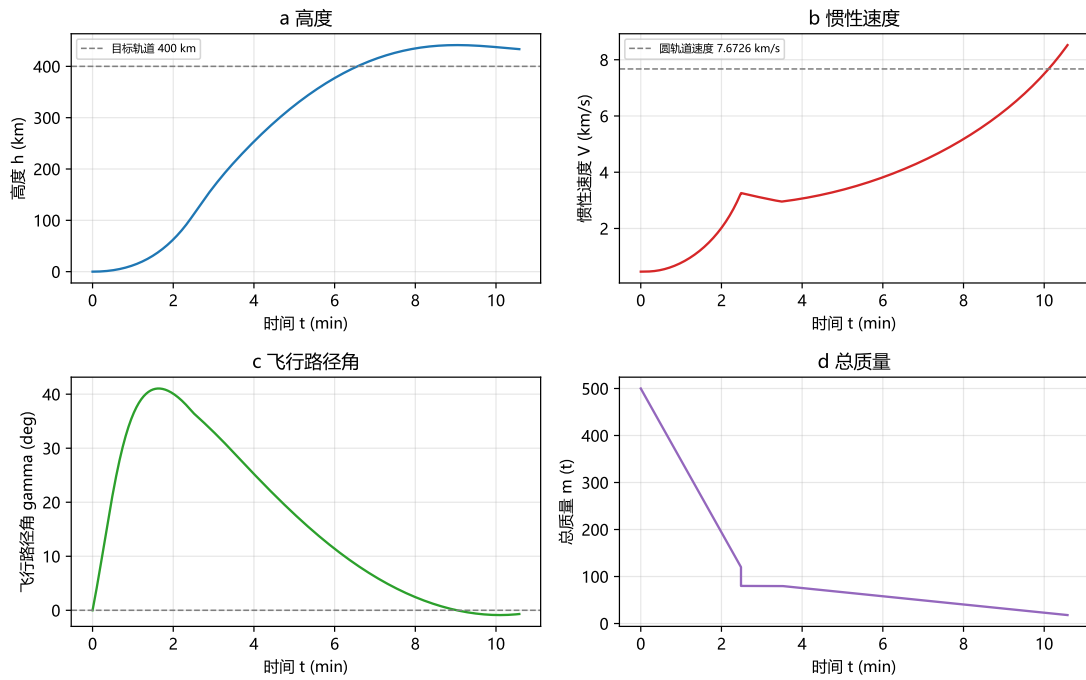


图 3 基准策略全程仿真曲线

结果解读：

1. 基准策略下二子级燃尽状态不在目标圆轨道终端流形上。由该状态反算得到半长轴约 8948 km、偏心率约 0.240，对应近地点约 431 km、远地点约 4723 km，说明它进入的是高偏心椭圆轨道，而非 400 km 圆轨道；
2. 这印证了题面的二子级可提前关机设定，必须通过调节滑行时间、俯仰角程序与提前关机实现精确入轨；

表 2 全程关键结果

量	一子级燃尽	二级点火	二级关机	圆轨道目标
时间 (s)	149.06	209.06	634.67	—
高度 (km)	109.26	210.90	433.53	400.00
惯性速度 (m/s)	3260.19	2954.53	8521.18	7672.60
路径角 (°)	36.61	29.30	-0.69	0
质量 (t)	120.0	80.0	18.0	—

3. 关机速度比目标圆轨道速度高约 849 m/s，只能作为存在提前关机潜力的定性证据；实际节省量还取决于推力方向、重力损失和关机位置，不能把速度差直接等同为推进剂节省；
4. 一子级关机点在 $h=109.26$ km、 $\gamma = 36.61^\circ$ ，说明转弯段经历强重力损失与阻力损失，是后续滑行与入轨段改进的物理起点。

5.7 理想速度增量分析

为量化各级实际效率，采用火箭方程计算真空理想速度增量，

$$\Delta v_{\text{ideal}} = I_{sp1}g_0 \ln \frac{M_0}{M_0 - m_{p1}} + I_{sp2}g_0 \ln \frac{m_{b2}}{m_{s2} + m_{pl}}, \quad (21)$$

其中 $m_{b2} = m_{s2} + m_{p2} + m_{pl}$ 为二子级点火质量。代入数值得到 $\Delta v_{\text{ideal}} = 10342$ m/s。而目标圆轨道惯性速度仅 7672.60 m/s，减去地球自转初速后理想增量需求为 7208 m/s。由此，两级发动机具备约 3134 m/s 的速度余量，该余量一部分被重力损失、气动阻力损失与转弯损失消耗，另一部分即提前关机留下的推进剂节约空间。这一能量关系从宏观上解释了问题二至问题四得以提前关机的物理基础，也为优化提供了判断量级。

六、 问题二：入轨条件三维打靶

6.1 入轨条件

目标为半径 $a = R_E + H$ 的圆轨道。圆轨道运动的特征由角动量与机械能决定：在中心引力场中，圆轨道要求火箭在关机时刻速度方向严格沿当地切向，即径向速度为零，角动量

$$h = |\mathbf{r} \times \mathbf{v}| = a^2 \dot{\theta} \quad (22)$$

保持不变；同时，在给定半径 a 处，圆轨道速度由引力与向心加速度平衡条件导出，即

$$\frac{\mu}{a^2} = \frac{v^2}{a} \implies v = \sqrt{\frac{\mu}{a}}. \quad (23)$$

因此火箭在关机时刻应满足目标半径、零径向速度与顺行切向速度三个条件。令 $v_r = \mathbf{v} \cdot \hat{\mathbf{r}}$ 、 $v_t = \mathbf{v} \cdot \hat{\mathbf{t}}$ ，则

$$|r(t_3)| = a, \quad v_r(t_3) = 0, \quad v_t(t_3) = +\sqrt{\mu/a}. \quad (24)$$

正号排除了同样满足速度大小与正交条件的逆行圆轨道。速度在惯性系中度量，与式 (2) 的惯性初速一致。图 4 给出入轨条件几何关系。

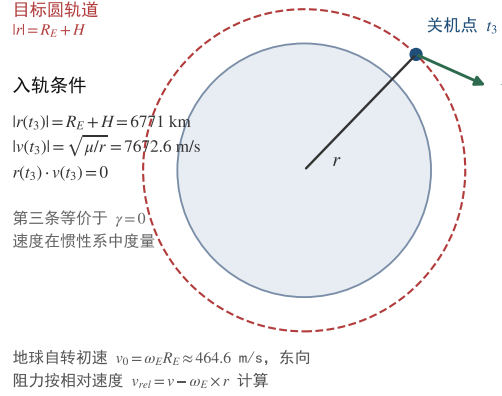


图 4 入轨条件

6.2 控制参数化

二子级俯仰角以恒定速率变化， $\phi(t) = \phi_0 + k(t - t_2)$ ，其中 ϕ_0 为点火时刻惯性速度路径角； k 为待求变化率；推力 $T = F_{\max 2}$ 恒定。为使边界清晰，以二级燃烧时间 $t_b = t_3 - t_2$ 代替绝对关机时刻，未知量写为

$$\mathbf{z} = [t_c, k, t_b]^T. \quad (25)$$

该参数化方式是问题二的核心：题目指定俯仰角恒定速率变化，故俯仰角律完全由两个量决定，其一为起始值 ϕ_0 ，其二为速率 k 。而 ϕ_0 由点火时刻状态唯一确定，既不由优化者控制也不独立，故决策变量集合中不含 ϕ_0 。

6.3 打靶模型

打靶法将两点边值问题化为初值问题迭代求解。给定 $\mathbf{z}$ ，从已知的发射初始状态出发，依次完成垂直段、程序转弯段、分离与滑行段积分，到达二子级点火点；再按俯仰角律 $\phi(t)$ 从点火点积分至关机时刻 t_3 ，得到关机状态。该状态与目标圆轨道之间的偏差即为入轨残差。

偏差三个分量量纲不同，直接合成会掩盖小量，因此采用无量纲化形式：

$$F(\mathbf{z}) = \left[\frac{|r| - a}{R_E}, \frac{v_r}{\sqrt{\mu/a}}, \frac{v_t - \sqrt{\mu/a}}{\sqrt{\mu/a}} \right]^T. \quad (26)$$

三个分量分别对应半径偏差、径向速度偏差与切向速度偏差，各自以其特征量归一化，使三者处于同一量级，便于数值求解。切向速度取顺行方向（局部东向），当 $v_t < +\sqrt{\mu/a}$ 时残差为负，从而严格排除逆行圆轨道分支。于是问题二归结为求非线性方程组

$$F(\mathbf{z}) = 0 \quad (27)$$

的根。这是三个未知量对应三个独立终端条件的方阵型闭合系统；根是否存在及是否唯一需要由数值搜索与后验复验判断，不能由变量个数直接保证。

6.4 有限维打靶与复验

一级轨迹只计算一次；对每个 t_c ，滑行段按二体动力学传播至二级点火，再积分二级动力段并评价 $F(\mathbf{z})$ 。胡冬生等指出，滑行时间受约束时，点火状态预报精度会显著影响边值迭代质量 [5]；在本文球形中心引力假设下，滑行段也可用 Kepler f - g 传播替代数值积分。数值实现对三个变量设置物理边界

$$0 \leq t_c \leq 400 \text{ s}, \quad -0.4 \leq k \leq 0.2^\circ/\text{s}, \quad 0 < t_b \leq t_{b2}, \quad (28)$$

并从网格多初值（196 组滑行时间、角速率与燃烧时间组合）出发作有界数值搜索。求得候选根后，用完整分段动力学重新传播并以米、米每秒量级的终端误差验收。求根器只是计算状态转移映射零点的工具，不构成该问的模型主体。

6.5 结果与分析

在给定搜索区间内得到一组通过独立复验的可行根，如表 3 所示。

表 3 圆轨道入轨打靶解

滑行时间	变化率	燃烧时间	关机时刻	初始俯仰角	剩余推进剂
103.632 s	-0.0474°/s	398.049 s	650.742 s	23.128°	4015 kg

入轨验证：高度 400.0000 km、惯性速度 7672.599 m/s、飞行路径角 0.0000°，残差达 10^{-10} 量级。结果解读：

1. 相比问题一的固定 60 s 滑行，该可行解的滑行时间增大至 103.63 s。更长的滑行使火箭在更高、更平坦的弹道上依靠重力完成转向，降低了二子级所需的方向修正量；
2. 俯仰角变化率为轻微负值，与点火后重力转向使 γ 自然减小的趋势匹配，避免过大攻角造成的推力浪费；
3. 提前关机留下 4015 kg 推进剂，正是问题一过冲的修正结果。

6.6 滑行时间对点火状态的影响分析

为理解最优滑行时间的分布规律，对滑行时间 0 至 300 s 扫描点火状态，结果如表 4 所示。

表 4 滑行时间的影响

滑行时间 s	点火高度 km	惯性速度 m/s	路径角 °	本地圆轨道速度差 m/s
0	109.26	3260.2	36.61	4582.6
30	163.82	3098.7	33.12	4711.3
60	210.90	2954.5	29.30	4827.5
113	276.10	2746.1	21.71	4997.7
200	334.25	2549.7	7.28	5160.4
250	340.52	2527.8	-1.64	5178.7
300	327.03	2574.7	-10.46	5139.6

观察发现：滑行时间从零增大时，点火高度持续上升，路径角单调下降，在约 250 s 处路径角过零并转为负值，对应椭圆滑行弹道的远地点。继续增大滑行时间后，火箭越过远地点并开始下降。该扫描说明滑行时间同时改变点火高度、速度方向和剩余重力损失；问题二的 103.63 s 只是给定线性俯仰律下的一组可行参数，放开控制函数后最优滑行时间可以显著改变。

七、 问题三：推力方向最优控制

7.1 连续模型及其结构

令 $t_b = t_3 - t_2$ 为二级燃烧时间。决策对象为滑行时间 t_c 、燃烧时间 t_b 、状态轨迹 $\mathbf{x}(t)$ 与俯仰角函数 $\phi(t)$ 。目标为最小化推进剂消耗；由于二级满推力且比冲恒定，目标等价于最小化燃烧时间（见下式）。约束分为三类：动力学方程、终端入轨条件与控制

量边界。综上，问题三的标准模型为

$$\begin{aligned} \min \quad & J = m(t_2^+) - m(t_3), \\ \text{s.t.} \quad & \begin{cases} \dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \phi(t), F_{\max 2}), \\ x(t_2^+) = x(t_2^-), \quad m(t_2^+) = m(t_2^-), \\ r(t_3) = a, \quad v_r(t_3) = 0, \quad v_t(t_3) = \sqrt{\mu/a}, \\ 0 \leq t_c \leq 400 \text{ s}, \quad 0 < t_b \leq t_{b2}, \\ -90^\circ \leq \phi(t) \leq 90^\circ, \\ m(t) \geq m_{s2} + m_{pl}. \end{cases} \end{aligned} \quad (29)$$

其中第一行为全程动力学约束，第二行为二子级点火时刻的状态连续条件（滑行段与动力段衔接），第三行为目标圆轨道终端流形，第四至六行为控制量与状态的可行域约束。二级满推力且比冲恒定，故

$$J = \frac{F_{\max 2}}{I_{sp2}g_0} t_b. \quad (30)$$

因此，推力方向不直接改变瞬时耗药率，而是通过改变终端可行性来缩短所需燃烧时间。式 (30) 是本问最重要的结构：燃料最优问题等价于自由终端时间的最短燃烧问题。

题目只在问题二规定二级点火初始俯仰角与速度同向，问题三、四未给出姿态角速度上限；结合质点假设，本文把 $\phi(t)$ 作为可直接选取的控制量。若只固定 $\phi(t_2)$ 而不限制 $|\dot{\phi}|$ ，细网格会在点火后的极短区间形成趋于跳变的边界层，该单点条件并不能代表真实姿态连续性。因此工程化扩展应引入姿态动力学或明确约束 $|\dot{\phi}| \leq \dot{\phi}_{\max}$ ，其数值取值需由题面或发动机摆角能力给定。

7.2 Hermite–Simpson 直接配点

将滑行段与二级动力段分别划分为 N_c 和 N_b 个区间，把每个节点的状态与控制同时作为决策变量。对任一长度为 Δt 的区间，设端点状态为 $\mathbf{x}_k, \mathbf{x}_{k+1}$ ，端点导数为 $\mathbf{f}_k, \mathbf{f}_{k+1}$ ，中点状态采用

$$\mathbf{x}_{k+\frac{1}{2}} = \frac{\mathbf{x}_k + \mathbf{x}_{k+1}}{2} + \frac{\Delta t}{8}(\mathbf{f}_k - \mathbf{f}_{k+1}), \quad (31)$$

并施加缺陷约束

$$\mathbf{x}_{k+1} - \mathbf{x}_k - \frac{\Delta t}{6}(\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} + \mathbf{f}_{k+1}) = \mathbf{0}. \quad (32)$$

由此，动力学不再隐藏在每一次目标函数评价的黑箱积分中，而是成为稀疏非线性规划的显式约束；阶段连接、质量下界和终端流形也以同一方式进入模型。该结构与 YAPSS Delta III 算例中的多阶段连续动力学、阶段连接和终端约束思想一致 [9]，但本文针对题设重新建立了两级质量、滑行时间和节流约束。直接法与打靶法的适用特点可参见 Betts 的综述 [3]，更高阶伪谱离散可作为扩展 [4]。

为改善尺度，以 a 、 $\sqrt{\mu/a}$ 和 80000 kg 分别无量纲化位置、速度与质量。用问题二的可行轨迹作为初值，利用 CasADi 自动微分和内点法求解式 (32) 形成的稀疏规划 [8]；求解结束后，将节点控制作分段线性插值，并用与配点过程独立的高精度积分器重新传播。这里数值算法只负责求解离散后的数学模型，可靠性由网格加密和独立复验共同判断。

配点求解的完整流程如下：

1. **建立决策变量：**滑行时间 t_c 、燃烧时间 t_b 、滑行段 $N_c + 1$ 个状态节点、动力段 $N_b + 1$ 个状态节点与 $N_b + 1$ 个俯仰角节点；
2. **施加约束：**一子级分离态固定为全部节点的初值；滑行段与动力段各自满足式 (32) 缺陷约束；动力段入口状态衔接滑行段出口；终端满足顺行圆轨道流形；
3. **无量纲化与初始化：**位置、速度、质量分别以 a 、 $\sqrt{\mu/a}$ 、80000 kg 归一；以问题二的可行轨迹插值作为状态与控制初值；
4. **求解：**先用粗网格 $(N_c, N_b) = (10, 20)$ 求解，再将其解插值到细网格 $(20, 40)$ 作为初值重解，并比较两网格的目标值差异；
5. **独立复验：**将所得控制律（节点分段线性插值）交给与配点完全独立的 DOP853 积分器重新传播，检查终端高度、径向速度与切向速度误差。

7.3 结果与网格一致性

表 5 给出两级网格结果。

表 5 问题三配点网格加密与独立复验

(N_c, N_b)	滑行时间/s	燃烧时间/s	耗药/kg	高度误差/m	$(\Delta v_r, \Delta v_t)/\text{m/s}$
(10, 20)	4.37665	394.35331	57446.923	-0.668	(-0.0022, -0.0015)
(20, 40)	4.37670	394.35339	57446.934	-0.919	(-0.0073, 0.0010)

两级网格耗药差为 0.012 kg，说明目标值已基本稳定；独立传播的终端高度误差小于 1 m，径向和切向速度误差均小于 0.01 m/s。最优解的关机时刻为 $t_3 = 547.791$ s，剩余推进剂 4553 kg。

为排除“网格稀导致目标值偏低”的可能，本文进一步用 11、21、31 节点三档网格（另一套独立配点实现，见支撑材料）交叉验证，结果如表 6 所示。

该独立实现在 11 到 31 节点间耗药仅差 0.18 kg（相对差 3×10^{-6} ），且 31 节点解与本文件 20/40 网格解（57446.934 kg）完全一致，证明结果与网格加密路径无关。值得注意的是，该实现采用固定步长四阶 Runge-Kutta 传播器而非 DOP853，与本文求解链完全独立，从另一条数值路径给出了相同的最优解。

表 6 问题三三档网格收敛（独立实现交叉验证）

节点数	滑行时间/s	燃烧时间/s	耗药/kg	独立复算终端残差
11	4.37571	394.35212	57446.750	4.5×10^{-3}
21	4.37659	394.35331	57446.923	2.9×10^{-4}
31	4.37386	394.35337	57446.933	5.7×10^{-5}

7.3.1 最优俯仰角控制律

图 5 给出最优解对应的完整飞行曲线，图 6 给出最优俯仰角控制律。需要强调，问题三的二子级发动机不具备节流能力、始终以额定满推力工作，俯仰角是唯一控制量。

问题3 最优轨迹（滑行 4.378 s + 燃烧 394.353 s，耗药 57446.9 kg）

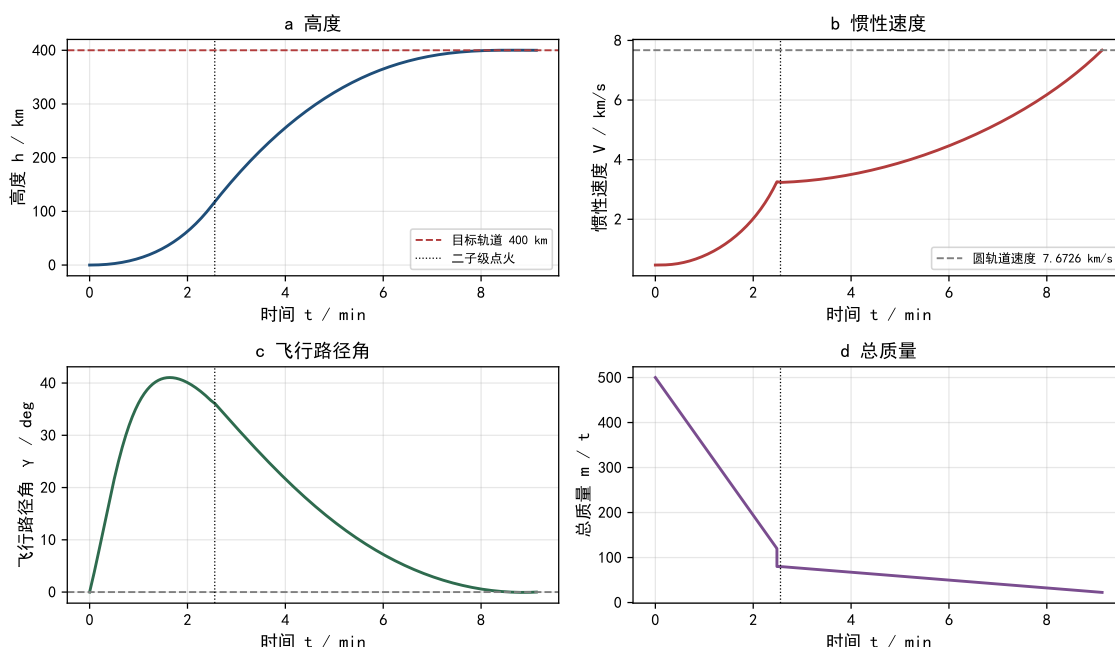


图 5 问题三最优解飞行曲线

由图 5 可见：二级点火时刻为 $t_2 = 153.438$ s，对应高度约 117.7 km、惯性速度约 3235.6 m/s、惯性飞行路径角约 36.1° 。点火后高度由约 118 km 连续上升至 400 km，惯性速度由约 3236 m/s 增长至圆轨道速度 7672.6 m/s，飞行路径角在点火后逐渐减小至零，总质量由 80 t 匀速下降至 22.6 t（耗药 57.4 t）。俯仰角控制律（图 6）呈“先降后升”的平滑曲线：由点火时 7.31° 平滑下降，在约 330 s 处达到极小值 1.75° ，随后缓慢回升至关机点 2.27° 。

需要强调，点火处最优推力俯仰角（ 7.31° ）与点火处惯性飞行路径角（ 36.1° ）显著不同——这正是问题三放开推力方向自由度后的结果：点火后推力方向无需与速度方向一致，而是以约 7.31° 的较平姿态切入，把推力更多用于补充切向速度，同时保留足够

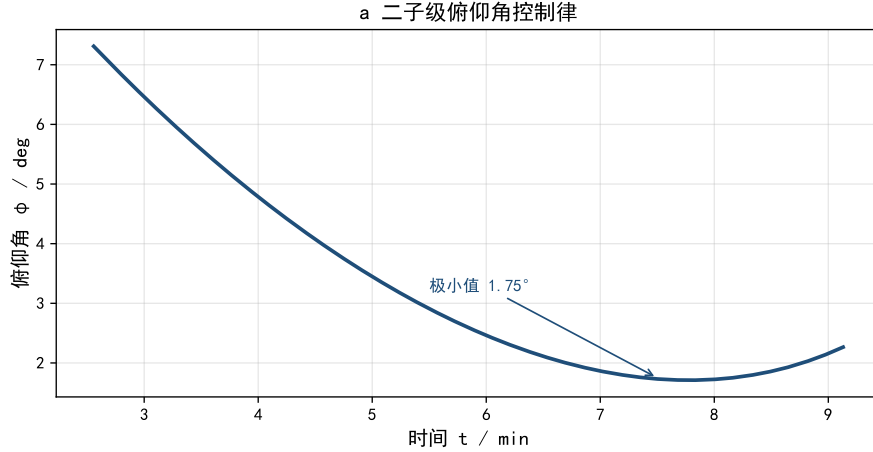


图 6 问题三最优俯仰角控制律

径向分量继续爬升。这一“先降后升”的形态可由终端条件解释：推力方向在前期略高于水平，以同时爬升并补足切向速度；中期贴近水平以最小化径向速度残差；末端小幅上抬以将径向速度精确清零、同时补足微小的切向速度缺口。

7.3.2 与其他方案的对比

与问题二线性俯仰律相比，本节结果耗药减少约 538.4 kg (57446.93 kg vs 57985.3 kg)。更重要的是，滑行时间从 103.63 s 改变为约 4.38 s。这表明旧的低维控制参数（恒定角速率）限制了可行轨迹族：在最优解中，二子级点火后即可通过连续调整推力方向完成上升段的重力转向，无需依赖长达百秒的无动力滑行来“等待”路径角归零。真正的收益来源于连续推力方向模型释放的控制自由度，而非更换求解器本身。

7.3.3 结果的有效性讨论

由于该问题非凸，直接配点只能保证局部最优。本文通过两项证据支持结果的可靠性：其一，网格加密后目标值变化仅 0.012 kg，说明离散误差已收敛；其二，控制律经独立 DOP853 积分复验，终端高度误差小于 1 m，说明该控制律对应真实动力学下的可行轨迹。因此本文将表中结果称为经过网格一致性与独立复验支持的局部最优候选解；若需要全局最优性保证，应进一步结合高阶伪谱离散与多起始点或同伦方法进行验证。

八、 问题四：推力节流下的燃料最省优化

8.1 优化模型建立

问题四在问题三的基础上允许二子级发动机推力在额定推力的 60%~100% 之间连续调节。按决策变量、目标函数、约束条件与标准模型的顺序建立模型。

8.1.1 决策变量

在问题三的决策变量基础上增加节流比控制变量。节流比 $\sigma(t)$ 是定义在燃烧段 $[t_2, t_3]$ 上的时间函数，与俯仰角 $\phi(t)$ 一样属于无限维控制量，配点转录后用 $N_b + 1$ 个节点值 $\sigma_0, \dots, \sigma_{N_b}$ 参数化。因此决策对象为：滑行时间 t_c 、燃烧时间 t_b 、状态轨迹 $\mathbf{x}(t)$ 、俯仰角函数 $\phi(t)$ 与节流比函数 $\sigma(t)$ 。

8.1.2 目标函数

节流改变的是瞬时推力与质量流量。由式 (14)，节流后的质量流量为

$$\dot{m}(t) = -\frac{\sigma(t)F_{\max 2}}{I_{sp2}g_0}, \quad (33)$$

故二子级推进剂消耗为

$$J = \int_{t_2}^{t_3} \frac{\sigma(t)F_{\max 2}}{I_{sp2}g_0} dt = m(t_2^+) - m(t_3). \quad (34)$$

与问题三不同，此时 J 不再简单地正比于燃烧时间 t_b ，而是正比于节流比对时间的积分：降低 σ 会减小单位时间耗药率，但同时需要延长燃烧时间才能达到相同的速度增量，两种效应相互竞争，最优节流律正是这种权衡的结果。

8.1.3 约束条件

问题四的约束条件与问题三基本相同，仅推进剂容量约束需要修正，分述如下。

1. 动力学约束。节流后的状态方程与式 (15)–(18) 形式一致，仅将推力 T 替换为 $T = \sigma(t)F_{\max 2}$ ：

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \phi(t), \sigma(t)). \quad (35)$$

2. 阶段连接约束。与问题三相同，二子级点火时刻状态连续，一子级分离态为固定初值。

3. 终端入轨条件。与问题三完全相同：

$$r(t_3) = a, \quad v_r(t_3) = 0, \quad v_t(t_3) = \sqrt{\mu/a}. \quad (36)$$

4. 推进剂容量约束（本问的关键修正）。在问题三中燃烧时间以满推力燃尽时长 $t_{b2} = 425.61$ s 为上界；节流后这一上界不再成立：例如恒定 60% 推力使用全部推进剂可工作 $t_{b2}/0.6 \approx 709.35$ s。正确的容量约束是累计耗药量不超过推进剂总量，即

$$\int_{t_2}^{t_3} \sigma(t) dt \leq t_{b2}, \quad \Longleftrightarrow \quad m(t_3) \geq m_{s2} + m_{pl} = 18000 \text{ kg}. \quad (37)$$

采用式 (37) 后，低推力长燃烧轨迹不会被人为排除。

5. 控制量边界约束。俯仰角可行域与问题三相同；节流比由发动机可稳定工作的推力范围给定，

$$0.6 \leq \sigma(t) \leq 1.0. \quad (38)$$

8.1.4 标准模型

综上，问题四的标准优化模型为

$$\begin{aligned} \min \quad & J = m(t_2^+) - m(t_3), \\ \text{s.t.} \quad & \begin{cases} \dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \phi(t), \sigma(t)), \\ x(t_2^+) = x(t_2^-), \quad m(t_2^+) = m(t_2^-), \\ r(t_3) = a, \quad v_r(t_3) = 0, \quad v_t(t_3) = \sqrt{\mu/a}, \\ \int_{t_2}^{t_3} \sigma(t) dt \leq t_{b2}, \quad m(t) \geq m_{s2} + m_{pl}, \\ 0.6 \leq \sigma(t) \leq 1.0, \quad -90^\circ \leq \phi(t) \leq 90^\circ, \\ 0 \leq t_c \leq 400 \text{ s}, \quad 0 < t_b \leq t_{b2}/0.6. \end{cases} \end{aligned} \quad (39)$$

其中第一行为动力学约束（含节流），第二行为点火点状态连续条件，第三行为目标圆轨道终端流形，第四行为推进剂容量约束（以累计推力积分形式给出），第五、六行为控制量与时间的可行域。

8.2 Hermite–Simpson 直接配点求解

问题四采用与问题三相同的 Hermite–Simpson 转录，仅在两处修改：其一，动力段右端函数 (32) 中的推力与质量流量替换为节流形式，即

$$T_k = \sigma_k F_{\max 2}, \quad \dot{m}_k = -\frac{\sigma_k F_{\max 2}}{I_{sp} 2g_0}, \quad k = 0, \dots, N_b; \quad (40)$$

其二，决策变量增加 $N_b + 1$ 个节流节点，并在约束中加入式 (37) 的离散形式

$$\frac{t_b}{N_b} \sum_{k=0}^{N_b-1} \frac{\sigma_k + \sigma_{k+1}}{2} \leq t_{b2}. \quad (41)$$

由于问题四放宽了燃烧时间上界（由 t_{b2} 放宽至 $t_{b2}/0.6$ ），初值选择对收敛性影响更大。求解流程如下：

1. 初值一（满推力暖启动）：直接取问题三的最优解， $\sigma_k \equiv 1.0$ ，燃烧时间 $t_b = 394.35$ s，作为第一个初值；
2. 初值二（低节流独立启动）：令 $\sigma_k \equiv 0.8$ ，燃烧时间相应延长至约 493 s，状态轨迹以问题三最优解插值并人为适应更长燃烧段，作为第二个初值；

3. **求解与网格加密**: 两类初值分别在 (10, 20) 网格求解, 再将各自解插值到 (20, 40) 网格重解, 比较目标值;
4. **独立复验**: 将最优节流与俯仰角律 (节点分段线性插值) 交与独立的 DOP853 积分器重新传播, 检查终端误差。

两类初值在同一物理可行域内求解, 避免依靠几十组盲目初值暴力搜索; 若两类初值收敛到不同解, 则可判断存在多个局部最优并考察其目标值差异。

8.3 结果与分析

表 7 给出两级网格结果。

表 7 问题四配点结果与独立复验

(N_c, N_b)	滑行时间/s	燃烧时间/s	节流范围	耗药/kg	高度误差/m
(10, 20)	4.37651	394.35350	0.999993–1	57446.923	−0.898
(20, 40)	4.37641	394.35381	0.999999–1	57446.935	−0.005

8.3.1 最优节流与俯仰角控制策略

图 7 与 8 给出最优解对应的完整飞行曲线与控制律。

问题4 最优轨迹 (滑行 4.377 s + 燃烧 394.354 s, 耗药 57446.9 kg)

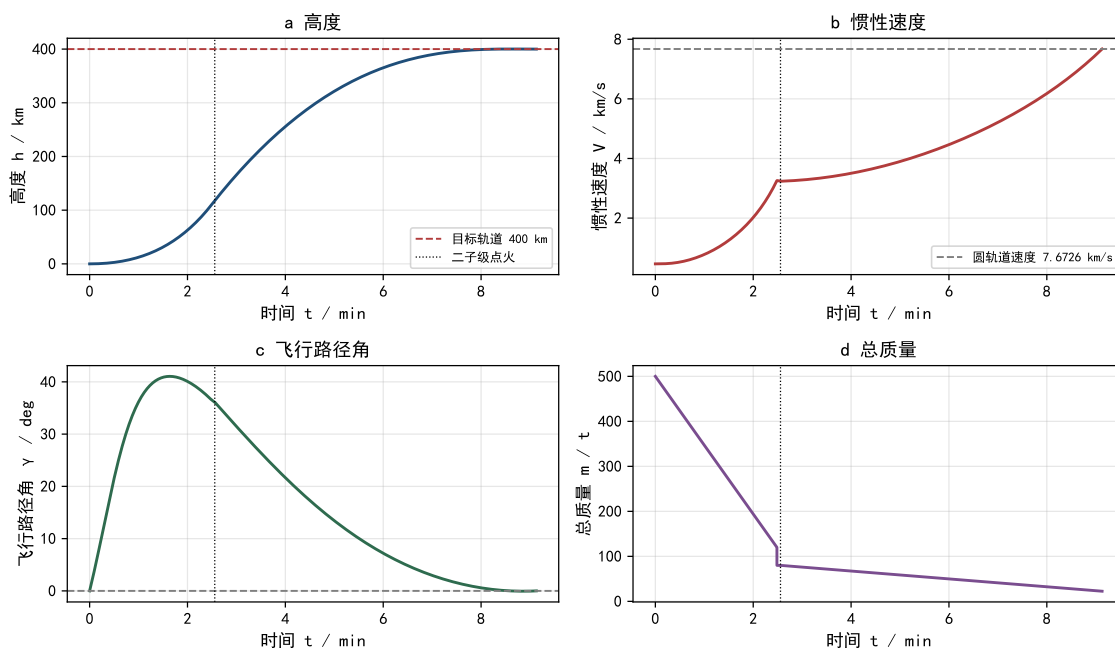


图 7 问题四最优解飞行曲线

最优控制策略可以完整表述为: 滑行时间 4.3764 s; 二子级以额定推力点火, 节流

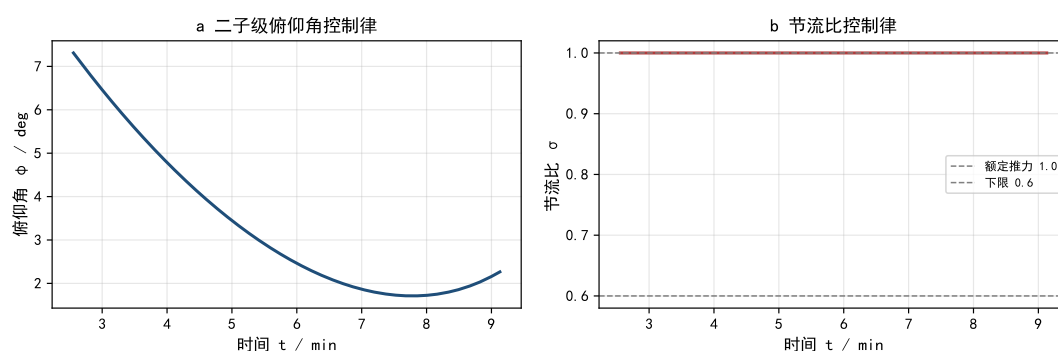


图 8 问题四最优控制律

比在全程保持在 $0.99998 \sim 1.00000$ 之间（图 8b 中 σ 曲线紧贴上限 1.0），即节流能力在工程可忽略的意义下未被使用；俯仰角由点火时的 7.31° 平滑下降，约 330 s 处达到极小 1.75° ，随后回升至关机点 2.27° 。与问题三的最优解相比，问题四仅把燃烧时间延长 0.0004 s、耗药增加 0.0001 kg——节流自由度对最优轨迹的贡献趋近于零。

8.3.2 两类初值的收敛性

以 $\sigma \equiv 0.8$ 、燃烧时间相应延长至约 493 s 的轨迹作为独立初值时，求解器同样收敛到上述近似满推力解（表 7 的第二行即从该初值出发的结果）。这说明结论对初值不敏感，不是靠初值引导出来的伪解。

为更严格地排除初值依赖，另一套独立配点实现以 0.6~1.0 五档节流常数分别作为初值求解，五组结果如表 8 所示。其中只有满推力初值直接落到 $\sigma \equiv 1$ ；其余四档初值收敛到节流节点在 0.60~0.99 与 1.0 之间的解，但其耗药量与满推力解仅差 0.0005~0.10 kg（相对差不足 2×10^{-6} ），且剩余节流节点均被优化推向 1.0 上界。该审计说明：即便允许从低节流初值出发，优化器也会把节流推向满推力，结论对初值稳健。

表 8 问题四多初值审计（独立实现）

节流初值	滑行时间/s	燃烧时间/s	耗药/kg	节流节点范围
1.0	4.37386	394.35337	57446.933	1.0~1.0
0.9	4.28268	394.44144	57446.934	0.987~1.0
0.8	3.80878	396.17906	57446.996	0.805~1.0
0.7	3.64006	397.33198	57447.022	0.656~1.0
0.6	3.40476	397.94467	57447.031	0.600~1.0

8.3.3 恒定节流灵敏度的定量结果

将节流比固定为常数（0.6~1.0 五档）并用配点法重新求解其余变量，得到燃料消耗随节流比的单调关系如表 9 所示。

表 9 问题四恒定节流灵敏度（独立实现）

恒定节流比	滑行时间/s	燃烧时间/s	耗药/kg
1.0	4.37386	394.35337	57446.933
0.9	0.00	438.52968	57494.035
0.8	0.00	494.62521	57643.125
0.7	0.00	568.58237	57979.266
0.6	0.00	671.89378	58726.370

在恒定节流这一控制族内，耗药随节流比单调增加：节流 0.9、0.8、0.7、0.6 分别使耗药增加 47.1、196.2、532.3、1279.4 kg。该定量结果直接量化了”降低推力延长燃烧时间、增加重力损失”的机理，与 §6 的理想速度增量分析（两级具备约 3134 m/s 速度余量）相互印证。需要说明的是，0.9~0.6 各档的最优滑行时间均收敛到 0 s 附近，说明低节流下”尽可能早点火、以长燃烧补足速度”成为局部最优形态；尽管如此，其耗药仍高于满推力解，进一步确认节流不带来收益。

8.3.4 速度损失预算

为进一步解释各方案的能量结构，表 10 给出理想速度增量与实际速度增量的分解。

表 10 二级段速度损失预算

方案	理想速度增量	实际速度增量	重力损失	转向损失
问题二（线性俯仰律）	5314.5	4894.2	367.1	53.3
问题三、四（最优俯仰律）	5215.0	4437.0	700.0	78.0

问题二的最优解以较长滑行换取较平缓的上升弧线，重力损失仅 367 m/s、转向损失 53 m/s；问题三的最优解几乎取消滑行、点火即开始大攻角转向，重力损失增大到 700 m/s，但总推力方向更贴合”最短燃烧时间”目标，其余量相互抵消后耗药反而更少。这一分解定量说明了问题的权衡结构：滑行时间并非越大越好，增长滑行期节省的重力损失会被更长的燃烧时间抵消。

8.3.5 物理解释与文献对照

在固定比冲模型中，降低推力会延长发动机在重力场中的工作时间。令 $\Delta v_{\text{eff}} = \Delta v_{\text{required}} + \Delta v_{\text{gravity}}$ ，其中 $\Delta v_{\text{gravity}} = \int g \sin \gamma dt$ 随燃烧时长近似线性增长，而燃烧时长

与节流比成反比，因此节流比越小、重力损失越大、所需速度增量越大、耗药越多（表 9 已给出定量对应）。本题又没有最大动压、过载或热流等迫使发动机降推力的路径约束，因而数值解落在满推力边界具有明确物理解释。

Nair 与 Vaidyanathan 的研究表明，节流的重要价值常来自满足路径约束并重塑整体弹道 [1]。本文结果与之一致：在无路径约束的设定下节流无损但亦无益。需要强调的是，本文的两种初值和两级网格仍只是局部数值证据，不能推出“无路径约束时节流必然无收益”的一般性定理；若要作此断言，还需对 Hamilton 函数关于 σ 的切换函数符号给出严格证明。若在题设中补充最大动压、过载或攻角约束，节流律可能出现“满推力—节流—满推力”的非平凡切换结构，届时应以本文配点框架重新求解，而不能沿用当前的满推力结论。

九、 模型评价

9.1 优点

1. 坐标系处理一致：运动方程写在地心惯性系中，地球自转同时进入初始速度与旋转大气相对速度，避免混用惯性速度和相对速度；
2. 四问共享同一多阶段动力学和顺行圆轨道终端流形，只逐步开放控制自由度，因而不同问题的结果可以直接比较；
3. 问题三揭示了“恒定满推力下最少燃料等价于最短燃烧时间”的结构，问题四则把容量条件正确提升为累计推力积分约束；
4. 配点控制均由独立初值问题积分器复验，并给出网格加密、米级高度误差和米每秒级速度误差，区分了离散模型可行性与真实动力学可行性。

9.2 缺点与改进

1. 平面假设忽略侧向运动，若目标轨道含倾角或发射场不在赤道，需扩展为三维模型；
2. 采用球形中心引力、指数大气和恒定比冲，地球半径取 6371 km；若改用赤道半径或加入 J_2 ，数值结果会发生系统变化，应做常数与模型敏感性分析；
3. 直接配点得到的是非凸规划的局部最优候选解。网格一致性与不同初值提高可信度，但不等同于全局最优证明；
4. 未考虑最大动压、过载、热流、俯仰角速度等路径约束。加入这些约束后，节流可能出现非平凡切换结构，应重新求解而不能沿用本问结论 [1, 2]；
5. 可进一步采用高斯伪谱法 [4] 或三维制导模型 [6] 进行交叉验证。

十、 结论

本文以统一的多阶段变质量动力学为主线回答四问。问题一表明基准策略将火箭送入高远地点椭圆轨道而非目标圆轨道；问题二在题设线性俯仰律下给出一组精确可行的三变量打靶解。问题三放开推力方向函数后，直接配点候选解将滑行时间缩短到约 4.38 s，耗药量降至 57446.93 kg，网格加密差异约 0.012 kg。问题四采用正确的累计耗药约束后，从满推力和低推力初值均收敛到近似满推力控制，在当前模型和网格下未发现节流收益。上述第三、四问结论均经独立高精度传播复验，但鉴于问题非凸，只作局部最优性陈述。该分层建模方式也符合运载火箭总体弹道设计由任务约束逐级落实到控制变量的基本思路 [7]。

参考文献

- [1] Nair V S, Vaidyanathan A. Ascent trajectory design and optimization of a two-stage throttleable liquid rocket[J]. *Advances in Space Research*, 2022, 69(12): 4358–4375.
- [2] Benedikter B, Zavoli A, Colasurdo G, et al. Convex optimization of launch vehicle ascent trajectory with heat-flux and splash-down constraints[J]. *Journal of Spacecraft and Rockets*, 2022, 59(3): 900–915.
- [3] Betts J T. Survey of numerical methods for trajectory optimization[J]. *Journal of Guidance, Control, and Dynamics*, 1998, 21(2): 193–207.
- [4] 刘超越, 张成. 基于高斯伪谱法的二级助推战术火箭多阶段轨迹优化 [J]. *兵工学报*, 2019, 40(2): 292–302.
- [5] 胡冬生, 刘楠, 童科伟, 等. 含滑行时间约束的真空段弹道设计研究 [J]. *宇航总体技术*, 2023, 7(1): 14–20.
- [6] 李惠峰, 张冉, 王嘉炜. 液体火箭上升段制导方法的发展综述 [J]. *航天控制*, 2023, 41(4): 3–12.
- [7] 余梦伦, 刘银, 张志国. 弹道学在运载火箭总体设计中的实践与展望 [J]. *宇航总体技术*, 2023, 7(2): 13–22.
- [8] Andersson J A E, Gillis J, Horn G, et al. CasADi: a software framework for nonlinear optimization and optimal control[J]. *Mathematical Programming Computation*, 2019, 11: 1–36. <https://web.casadi.org/docs/#optimal-control-with-casadi>.
- [9] YAPSS Documentation. Delta III launch vehicle ascent problem[EB/OL]. https://yapss.readthedocs.io/en/stable/notebooks/delta_iii_ascent.html.

A 支撑材料文件列表

本论文的全部建模、求解与验证程序代码随支撑材料一并提交，完整源代码见附录 A–F，文件列表如下：

- `src/common.py`: 物理常数、火箭参数、动力学方程、分段积分器与入轨残差（附录 A）；
- `src/q1_baseline.py`: 问题一基准控制策略全程仿真（附录 B）；
- `src/q2_inscription.py`: 问题二入轨条件有界三维打靶（附录 C）；
- `src/q34_direct_collocation.py`: 问题三、四 Hermite–Simpson 直接配点求解器（附录 D）；
- `src/run_all.py`: 一键复现脚本（附录 E）；
- `cross_validation/`: 独立实现（RK4 + `scipy` 配点）的交叉验证代码（附录 F）、结果与说明（三档网格收敛、多初值审计、节流灵敏度、损失预算）；
- `tests/`: 配点数学检查与模型约束回归测试（含顺行圆轨道分支测试）；
- `results/`: 全部权威数值结果（含配点网格收敛与独立重积分复验数据）。

全部程序可通过 `python src/run_all.py` 一键复现论文数值结果与图表。

B 全程仿真公共模块 common.py

```
1 """两级运载火箭发射轨迹建模：公共工具模块（问题 1~4 共用）。
2
3 提供：物理常数、火箭参数、指数大气模型、平面点质量动力学（惯性 ECI 分量，
4 含地球自转与大气阻力）、分段数值积分器、入轨条件残差、绘图与结果输出。
5 """
6
7 from __future__ import annotations
8
9 from dataclasses import dataclass, field
10 from pathlib import Path
11
12 import numpy as np
13 import pandas as pd
14 from scipy.integrate import solve_ivp
15
16 # -----
17 # 物理常数（SI）
18 # -----
19 MU = 3.986004418e14          # 地球引力常数 [m^3/s^2]
20 R_E = 6371.0e3              # 地球半径（平均）[m]
21 OMEGA_E = 7.2921159e-5      # 地球自转角速度 [rad/s]
22 G0 = 9.80665                # 海平面重力加速度 [m/s^2]
23 RH00 = 1.225                 # 海平面大气密度 [kg/m^3]
24 H_SCALE = 7200.0            # 指数大气标高 [m]
25
26 ROOT = Path(__file__).resolve().parent.parent
27 FIG_DIR = ROOT / "figures"
28 RES_DIR = ROOT / "results"
29 FIG_DIR.mkdir(exist_ok=True)
30 RES_DIR.mkdir(exist_ok=True)
31
32 DEG = np.pi / 180.0
33
34
35 # -----
36 # 火箭参数（题目给定）
37 # -----
38 @dataclass
39 class RocketParams:
40     """两级液体运载火箭参数（题面给定值）。"""
41
42     # 一子级
43     M0: float = 500_000.0      # 起飞质量（含有效载荷与二子级）[kg]
44     ms1: float = 40_000.0     # 一子级结构质量 [kg]
45     mp1: float = 380_000.0     # 一子级推进剂质量 [kg]
```

```

46     Isp1: float = 300.0          # 真空比冲 [s]
47     Fmax1: float = 7.5e6        # 额定最大推力 [N]
48     S: float = 12.5            # 气动阻力参考面积 [m^2]
49     CD: float = 0.3            # 阻力系数 (忽略马赫数变化)
50     # 二子级
51     ms2: float = 8_000.0        # 二子级结构质量 [kg]
52     mp2: float = 62_000.0       # 二子级推进剂质量 [kg]
53     m_payload: float = 10_000.0 # 有效载荷质量 [kg]
54     Isp2: float = 420.0         # 真空比冲 [s]
55     Fmax2: float = 6.0e5        # 额定最大推力 [N]
56     # 任务
57     H_target: float = 400.0e3    # 目标圆轨道高度 [m]
58     t_vertical: float = 10.0     # 垂直起飞时长 [s]
59     pitch_rate_q1: float = 0.4 * DEG # 问题 1 程序转弯俯仰角角速率 [rad/s]
60     t_coast_q1: float = 60.0     # 问题 1 滑行时间 [s]
61
62     @property
63     def mdot1(self) -> float:
64         """一子级质量流量 =  $F/(Isp \cdot g_0)$  (真空、全推力) [kg/s]。"""
65         return self.Fmax1 / (self.Isp1 * G0)
66
67     @property
68     def mdot2(self) -> float:
69         """二子级额定质量流量 [kg/s]。"""
70         return self.Fmax2 / (self.Isp2 * G0)
71
72     @property
73     def t_burn1(self) -> float:
74         """一子级推进剂耗尽时间 [s] (一级全推力恒额定, 故为定值)。"""
75         return self.mp1 / self.mdot1
76
77     @property
78     def t_burn2(self) -> float:
79         """二子级推进剂耗尽时间 [s]。"""
80         return self.mp2 / self.mdot2
81
82     @property
83     def m_after_sep(self) -> float:
84         """一子级分离瞬间 (关机后) 剩余质量 [kg]:  $M_0 - mp1 - ms1$ 。"""
85         return self.M0 - self.mp1 - self.ms1
86
87     @property
88     def a_target(self) -> float:
89         """目标圆轨道地心距 [m]。"""
90         return R_E + self.H_target
91
92     @property
93     def v_circular(self) -> float:

```

```

94     """目标圆轨道惯性速度  $\sqrt{\mu/a}$  [m/s]。"""
95     return np.sqrt(MU / self.a_target)
96
97
98 # -----
99 # 大气模型
100 # -----
101 def rho_air(h: np.ndarray | float) -> np.ndarray | float:
102     """指数大气密度模型  $\rho = \rho_0 * \exp(-h / h_0)$ 。"""
103     return RH00 * np.exp(-h / H_SCALE)
104
105
106 # -----
107 # 平面点质量动力学（惯性 ECI 笛卡尔分量，赤道平面内）
108 # -----
109 def rel_velocity(rx: np.ndarray, ry: np.ndarray, vx: np.ndarray, vy: np.ndarray):
110     """相对大气速度  $\mathbf{v}_{rel} = \mathbf{v}_{in} - \boldsymbol{\omega}_E \times \mathbf{r}$ （赤道平面： $\boldsymbol{\omega}_E \times \mathbf{r} = \omega * [-y, x]$ 
    ↪ ）。"""
111     return vx + OMEGA_E * ry, vy - OMEGA_E * rx
112
113
114 def flight_path_angle(vrx: np.ndarray, vry: np.ndarray, rx: np.ndarray, ry: np.
    ↪ ndarray):
115     """相对速度飞行路径角  $\gamma = \text{atan2}(\mathbf{v} \cdot \mathbf{r}_{hat}, \mathbf{v} \cdot \mathbf{t}_{hat})$  [rad]。
116
117      $\mathbf{r}_{hat} = \mathbf{r}/|\mathbf{r}|$ ,  $\mathbf{t}_{hat} = \mathbf{z}_{hat} \times \mathbf{r}_{hat}$ （东向）。
118     垂直上升段  $v \sim 0$  时输出无定义，入轨段（高空）有效。
119     """
120     r = np.hypot(rx, ry)
121     r_dot = (vrx * rx + vry * ry) / r # 径向分量
122     t_dot = (vrx * (-ry) + vry * rx) / r # 切向（东向）分量
123     return np.arctan2(r_dot, t_dot)
124
125
126 def gamma_inertial(vx: np.ndarray, vy: np.ndarray, rx: np.ndarray, ry: np.ndarray):
127     """惯性速度飞行路径角（与入轨条件  $\mathbf{r} \cdot \mathbf{v} = 0$  直接对应）[rad]。"""
128     r = np.hypot(rx, ry)
129     r_dot = (vx * rx + vy * ry) / r
130     t_dot = (vx * (-ry) + vy * rx) / r
131     return np.arctan2(r_dot, t_dot)
132
133
134 def drag_accel(
135     vrx: np.ndarray, vry: np.ndarray,
136     rx: np.ndarray, ry: np.ndarray,
137     S: float, CD: float,
138 ) -> tuple[np.ndarray, np.ndarray]:
139     """气动阻力加速度 [m/s^2]:  $\mathbf{a}_D = -0.5 * \rho * S * CD * |\mathbf{v}_{rel}| * \mathbf{v}_{rel} / m$ 。

```

```

140
141     返回 (ax, ay) (惯性分量)。此处不含质量，由调用者再除以 m。
142     """
143     r = np.hypot(rx, ry)
144     h = r - R_E
145     rho = rho_air(h)
146     vrel = np.hypot(vrx, vry)
147     q = 0.5 * rho * S * CD * vrel
148     return -q * vr_x, -q * vr_y
149
150
151 def thrust_unit(phi: np.ndarray, rx: np.ndarray, ry: np.ndarray):
152     """由俯仰角 phi (推力与当地水平面的夹角) 构造推力单位向量  $\hat{u}$  (惯性分量)。
153
154      $\hat{u} = \cos(\phi) * \hat{t} + \sin(\phi) * \hat{r}$ 。
155     """
156     r = np.hypot(rx, ry)
157     rx_h, ry_h = rx / r, ry / r          # 径向单位向量
158     tx_h, ty_h = -ry / r, rx / r         # 东向单位向量
159     return tx_h * np.cos(phi) + rx_h * np.sin(phi), ty_h * np.cos(phi) + ry_h * np.
    ↪ sin(phi)
160
161
162 # -----
163 # 分段数值积分器
164 # -----
165 @dataclass
166 class Segment:
167     """一段飞行轨迹的积分结果。"""
168
169     t: np.ndarray
170     x: np.ndarray    # [x, y, vx, vy, m]
171     vr_x: np.ndarray
172     vr_y: np.ndarray
173     gamma: np.ndarray    # 惯性速度飞行路径角
174     gamma_rel: np.ndarray # 相对速度飞行路径角
175     phi: np.ndarray    # 俯仰角
176     T: np.ndarray    # 推力
177     sigma: np.ndarray
178     name: str = ""
179
180     @property
181     def h(self) -> np.ndarray:
182         return np.hypot(self.x[:, 0], self.x[:, 1]) - R_E
183
184     @property
185     def v_in(self) -> np.ndarray:
186         return np.hypot(self.x[:, 2], self.x[:, 3])

```

```

187
188 @property
189 def m(self) -> np.ndarray:
190     return self.x[:, 4]
191
192 @property
193 def time(self) -> np.ndarray:
194     return self.t
195
196
197 @dataclass
198 class SimResult:
199     """完整仿真结果（各阶段拼接）。"""
200
201     t: np.ndarray
202     x: np.ndarray          # 分段拼接后的状态
203     vrx: np.ndarray
204     vry: np.ndarray
205     gamma: np.ndarray      # 惯性速度飞行路径角
206     gamma_rel: np.ndarray  # 相对速度飞行路径角
207     phi: np.ndarray
208     T: np.ndarray
209     sigma: np.ndarray
210     phase: list[str] = field(default_factory=list) # 每点的阶段名
211     t1: float | None = None      # 一子级燃尽时刻
212     t2: float | None = None      # 二子级点火时刻
213     t_shut: float | None = None  # 二子级关机（实际）时刻
214     stages: list[Segment] = field(default_factory=list)
215
216 @property
217 def h(self) -> np.ndarray:
218     return np.hypot(self.x[:, 0], self.x[:, 1]) - R_E
219
220 @property
221 def v_in(self) -> np.ndarray:
222     return np.hypot(self.x[:, 2], self.x[:, 3])
223
224 @property
225 def m(self) -> np.ndarray:
226     return self.x[:, 4]
227
228 @property
229 def gamma_deg(self) -> np.ndarray:
230     return self.gamma / DEG
231
232 def final_state(self) -> dict:
233     return {
234         "h_km": float(np.hypot(self.x[-1, 0], self.x[-1, 1]) / 1e3 - R_E / 1e3)

```

```

↪ ,
235         "v_in_mps": float(np.hypot(self.x[-1, 2], self.x[-1, 3])),
236         "gamma_deg": float(self.gamma[-1] / DEG),
237         "gamma_rel_deg": float(self.gamma_rel[-1] / DEG),
238         "m_kg": float(self.x[-1, 4]),
239     }
240
241
242 class Simulator:
243     """飞行段积分器：垂直段 -> 程序转弯段 -> 分离 -> 滑行段 -> 二级动力段。"""
244
245     def __init__(self, rk: RocketParams | None = None):
246         self.rk = rk or RocketParams()
247
248     # -- 各段右端函数 -----
249     def _rhs(
250         self,
251         t: float, x: np.ndarray,
252         T_const: float, sigma: float,
253         phi: float, along_vel: bool,
254         drag_S: float, drag_CD: float,
255         Isp: float,
256     ) -> np.ndarray:
257         rx, ry, vx, vy, m = x
258         T = T_const * sigma
259         if along_vel:
260             vr_x, vr_y = rel_velocity(rx, ry, vx, vy)
261             vrel = np.hypot(vr_x, vr_y)
262             tx_h, ty_h = (vr_x / vrel, vr_y / vrel) if vrel > 1e-9 else (0.0, 1.0)
263         else:
264             tx_h, ty_h = thrust_unit(phi, rx, ry)
265         r = np.hypot(rx, ry)
266         ax = -MU / r**3 * rx + T / m * tx_h
267         ay = -MU / r**3 * ry + T / m * ty_h
268         if drag_S > 0.0:
269             vr_x, vr_y = rel_velocity(rx, ry, vx, vy)
270             dax, day = drag_accel(vr_x, vr_y, rx, ry, drag_S, drag_CD)
271             ax += dax / m
272             ay += day / m
273         return np.array([vx, vy, ax, ay, -T / (Isp * G0)])
274
275     # -- 单段积分 -----
276     def _integrate(
277         self,
278         t0: float, tf: float, y0: np.ndarray,
279         T_const: float, sigma: float,
280         phi_fn, along_vel: bool,
281         drag_S: float, drag_CD: float,

```

```

282     rtol: float, atol: np.ndarray,
283     name: str,
284 ) -> Segment:
285     def rhs(t, x):
286         phi = phi_fn(t) if phi_fn is not None else 0.0
287         return self._rhs(
288             t, x, T_const, sigma, phi, along_vel, drag_S, drag_CD, self.rk.Isp1
289         )
290
291     sol = solve_ivp(
292         rhs, (t0, tf), y0, method="DOP853",
293         rtol=rtol, atol=atol, dense_output=True,
294     )
295     t = np.linspace(t0, tf, max(2, int((tf - t0) * 20) + 1))
296     x = sol.sol(t).T
297     vrx, vry = rel_velocity(x[:, 0], x[:, 1], x[:, 2], x[:, 3])
298     gamma = gamma_inertial(x[:, 2], x[:, 3], x[:, 0], x[:, 1])
299     gamma_rel = flight_path_angle(vrx, vry, x[:, 0], x[:, 1])
300     if along_vel:
301         phi = np.where(
302             np.hypot(vrx, vry) > 1e-9,
303             np.arctan2(vrx * x[:, 0] + vry * x[:, 1],
304                       vrx * (-x[:, 1]) + vry * x[:, 0]),
305             np.nan,
306         )
307     else:
308         phi = np.array([phi_fn(ti) if phi_fn is not None else np.nan for ti in
309 ↪ t])
310     T = np.full_like(t, T_const * sigma)
311     return Segment(t, x, vrx, vry, gamma, gamma_rel, phi, T, np.full_like(t,
312 ↪ sigma), name)
313
314 # -- 全流程 -----
315 def simulate(
316     self,
317     t_coast: float = 60.0,
318     controller=None,
319     t_shut: float | None = None,
320     sigma: float | callable = 1.0,
321     rtol: float = 1e-9,
322     drag_after_sep: bool = False,
323 ) -> SimResult:
324     """按六段剖面仿真。
325
326     controller(t) -> phi [rad]: 二级动力段俯仰角程序; None 表示攻角为零
327     (真空二级段推力方向与惯性速度方向一致)。
328     sigma: 二级节流比 (常数或函数 t->sigma, 默认 1.0)。
329     t_shut: 二级关机时刻 (绝对时间)。None 表示按满推力燃尽时间关机;

```



```

328     若给定更晚时刻，则由于质量事件限制累计推进剂消耗。
329     返回 SimResult（全阶段拼接）。
330     """
331     rk = self.rk
332     st = []
333     atol = np.array([1e-2, 1e-2, 1e-4, 1e-4, 1e-2])
334
335     # --- 1) 垂直起飞段 [0, t_vert]: 推力沿当地径向，全推力，有阻力 ---
336     y0 = np.array([R_E, 0.0, 0.0, OMEGA_E * R_E, rk.M0])
337     phi = 0.5 * np.pi
338     seg = self._integrate(
339         0.0, rk.t_vertical, y0,
340         rk.Fmax1, 1.0, lambda t: phi, False,
341         rk.S, rk.CD, rtol, atol, "vertical",
342     )
343     st.append(seg)
344
345     # --- 2) 程序转弯段 [t_vert, t_burn1]: phi 以 0.4deg/s 线性减小 ---
346     phi_fn = lambda t: 0.5 * np.pi - rk.pitch_rate_q1 * (t - rk.t_vertical)
347     seg = self._integrate(
348         rk.t_vertical, rk.t_burn1, seg.x[-1],
349         rk.Fmax1, 1.0, phi_fn, False,
350         rk.S, rk.CD, rtol, atol, "pitch-over",
351     )
352     st.append(seg)
353
354     # --- 3) 一子级分离（质量跳变） ---
355     x1 = seg.x[-1].copy()
356     x1[4] -= rk.ms1
357
358     # --- 4) 无动力滑行段 [t1, t1+t_coast]: 无阻力 ---
359     t1 = rk.t_burn1
360     t2 = t1 + t_coast
361     seg = self._integrate(
362         t1, t2, x1,
363         0.0, 1.0, lambda t: 0.0, False,
364         0.0, 0.0, rtol, atol, "coast",
365     )
366     st.append(seg)
367
368     # --- 5) 二级动力段 [t2, t_shut] ---
369     mdot2 = rk.mdot2
370     sigma_fn = sigma if callable(sigma) else (lambda t: float(sigma))
371     if t_shut is None:
372         t_shut = t2 + rk.t_burn2 # 燃尽
373
374     def mass_flow(t):
375         return -sigma_fn(t) * rk.Fmax2 / (rk.Isp2 * G0)

```

```

376
377 def rhs2(t, x):
378     sig = sigma_fn(t)
379     T = sig * rk.Fmax2
380     if controller is None:
381         # 推力方向与惯性速度方向一致（二子级段无大气阻力，无相对大气参照）
382         vx, vy = x[2], x[3]
383         vmag = np.hypot(vx, vy)
384         tx_h, ty_h = (vx / vmag, vy / vmag) if vmag > 1e-9 else (0.0, 1.0)
385     else:
386         p = controller(t)
387         tx_h, ty_h = thrust_unit(p, x[0], x[1])
388     r = np.hypot(x[0], x[1])
389     ax = -MU / r**3 * x[0] + T / x[4] * tx_h
390     ay = -MU / r**3 * x[1] + T / x[4] * ty_h
391     return np.array([x[2], x[3], ax, ay, mass_flow(t)])
392
393 sol = solve_ivp(
394     rhs2, (t2, t_shut), seg.x[-1], method="DOP853",
395     rtol=rtol, atol=atol,
396     events=[lambda t, x: x[4] - (rk.ms2 + rk.m_payload)],
397     dense_output=True,
398 )
399 if sol.t_events[0].size > 0:
400     t_shut = float(sol.t_events[0][0])
401     tt = np.linspace(t2, t_shut, max(2, int((t_shut - t2) * 20) + 1))
402     xs = sol.sol(tt).T
403     sig_arr = np.array([sigma_fn(ti) for ti in tt])
404     vr_x, vr_y = rel_velocity(xs[:, 0], xs[:, 1], xs[:, 2], xs[:, 3])
405     gamma = gamma_inertial(xs[:, 2], xs[:, 3], xs[:, 0], xs[:, 1])
406     gamma_rel = flight_path_angle(vr_x, vr_y, xs[:, 0], xs[:, 1])
407     if controller is None:
408         phi_arr = np.where(
409             np.hypot(xs[:, 2], xs[:, 3]) > 1e-9,
410             np.arctan2(xs[:, 2] * xs[:, 0] + xs[:, 3] * xs[:, 1],
411                      xs[:, 2] * (-xs[:, 1]) + xs[:, 3] * xs[:, 0]),
412             np.nan,
413         )
414     else:
415         phi_arr = np.array([controller(ti) for ti in tt])
416     T_arr = sig_arr * rk.Fmax2
417     seg2 = Segment(tt, xs, vr_x, vr_y, gamma, gamma_rel, phi_arr, T_arr, sig_arr,
418 ↪ "stage2")
419     st.append(seg2)
420
421 # --- 拼接 ---
422 t = np.concatenate([s.t for s in st])
423 x = np.vstack([s.x for s in st])

```

```

423     vrx = np.concatenate([s.vrx for s in st])
424     vry = np.concatenate([s.vry for s in st])
425     gamma = np.concatenate([s.gamma for s in st])
426     gamma_rel = np.concatenate([s.gamma_rel for s in st])
427     phi = np.concatenate([s.phi for s in st])
428     T = np.concatenate([s.T for s in st])
429     sig = np.concatenate([s.sigma for s in st])
430     names = []
431     for s in st:
432         names += [s.name] * len(s.t)
433     return SimResult(t, x, vrx, vry, gamma, gamma_rel, phi, T, sig, names,
434                     t1, t2, t_shut, st)
435
436
437 # -----
438 # 入轨条件残差（问题 2~4 共用）
439 # -----
440 def orbit_residual(xf: np.ndarray, rk: RocketParams) -> np.ndarray:
441     """由关机时刻惯性状态计算入轨条件残差（无量纲，顺行圆轨道）。
442
443     三个分量依次为：半径偏差、径向速度偏差、切向速度偏差。
444     切向速度取顺行方向（局部东向），当  $v_t < v_c$  时残差为负，
445     从而严格排除逆行圆轨道分支。
446     xf = [x, y, vx, vy, m]（惯性）。
447     """
448     r = np.hypot(xf[0], xf[1])
449     rx, ry = xf[0] / r, xf[1] / r           # 径向单位向量
450     tx, ty = -ry, rx                       # 东向（顺行）单位向量
451     vr = xf[2] * rx + xf[3] * ry
452     vt = xf[2] * tx + xf[3] * ty
453     a = rk.a_target
454     vc = rk.v_circular
455     return np.array([
456         (r - a) / R_E,
457         vr / vc,
458         (vt - vc) / vc,
459     ])
460
461
462 def fmt_residual(res: np.ndarray) -> str:
463     return "[" + ", ".join(f"{v:+.2e}" for v in res) + "]"

```

C 问题一基准控制策略全程仿真 q1_baseline.py

```

1 """问题 1：基准控制策略下的全程动力学仿真。
2

```

```

3  基准策略（题目给定）：
4  - 一子级：额定最大推力；垂直起飞 10 s 后俯仰角以 0.4deg/s 线性减小（程序转弯），
5    直至推进剂耗尽关机分离；
6  - 二子级：全额推力，推力方向始终与（相对大气）速度方向一致（恒攻角为零）；
7  - 滑行时间 60 s。
8  输出：起飞至二子级燃料耗尽全程的高度、速度、飞行路径角、总质量曲线，
9  以及二子级关机时刻的轨道高度、惯性速度与飞行路径角。
10 """
11
12 from __future__ import annotations
13
14 import sys
15 from pathlib import Path
16
17 import matplotlib
18
19 matplotlib.use("Agg")
20 import matplotlib.pyplot as plt
21 import numpy as np
22 import pandas as pd
23
24 from common import (
25     DEG,
26     R_E,
27     RES_DIR,
28     FIG_DIR,
29     RocketParams,
30     Simulator,
31     orbit_residual,
32 )
33
34 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial"]
35 plt.rcParams["axes.unicode_minus"] = False
36
37
38 def phase_boundaries(res) -> dict:
39     """各阶段边界时刻（供绘图竖线）。"""
40     return {
41         "垂直段结束": res.stages[0].t[-1],
42         "一子级关机": res.t1,
43         "二子级点火": res.t2,
44         "二子级关机": res.t_shut,
45     }
46
47
48 def plot_profiles(res: SimResult, rk: RocketParams, fname: str) -> None:
49     """四联图：高度、惯性速度、飞行路径角、总质量（含阶段边界竖线）。"""
50     fig, axes = plt.subplots(2, 2, figsize=(12, 8))

```

```

51 ax = axes[0, 0]
52 ax.plot(res.t / 60.0, res.h / 1e3, lw=1.6, color="tab:blue")
53 ax.set_xlabel("时间 t (min)")
54 ax.set_ylabel("高度 h (km)")
55 ax.set_title("(a) 飞行高度")
56 ax.grid(alpha=0.4)
57
58
59 ax = axes[0, 1]
60 ax.plot(res.t / 60.0, res.v_in / 1e3, lw=1.6, color="tab:red")
61 ax.axhline(rk.v_circular / 1e3, ls="--", lw=1.0, color="gray",
62           label=f"目标圆轨道速度 {rk.v_circular/1e3:.1f} km/s")
63 ax.set_xlabel("时间 t (min)")
64 ax.set_ylabel("惯性速度 V (km/s)")
65 ax.set_title("(b) 惯性速度")
66 ax.legend(fontsize=8)
67 ax.grid(alpha=0.4)
68
69 ax = axes[1, 0]
70 ax.plot(res.t / 60.0, res.gamma / DEG, lw=1.6, color="tab:green", label="惯性
71 ↪ gamma")
72 ax.plot(res.t / 60.0, res.gamma_rel / DEG, lw=1.2, ls="--", color="tab:orange",
73         label="相对大气 gamma")
74 ax.set_xlabel("时间 t (min)")
75 ax.set_ylabel("飞行路径角 gamma (deg)")
76 ax.set_title("(c) 飞行路径角")
77 ax.legend(fontsize=8)
78 ax.grid(alpha=0.4)
79
80 ax = axes[1, 1]
81 ax.plot(res.t / 60.0, res.m / 1e3, lw=1.6, color="tab:purple")
82 ax.set_xlabel("时间 t (min)")
83 ax.set_ylabel("总质量 m (t)")
84 ax.set_title("(d) 总质量")
85 ax.grid(alpha=0.4)
86
87 for ax in axes.ravel():
88     for name, tb in phase_boundaries(res).items():
89         if tb is not None:
90             ax.axvline(tb / 60.0, ls=":", lw=0.8, color="k", alpha=0.5)
91 fig.suptitle("问题 1: 基准控制策略全程飞行曲线", fontsize=13)
92 fig.tight_layout(rect=(0, 0, 1, 0.96))
93 fig.savefig(FIG_DIR / fname, dpi=150)
94 plt.close(fig)
95
96 def plot_trajectory(res: SimResult, fname: str) -> None:
97     """地面轨迹（经度 vs 高度）与轨道图。"""

```

```

98     rx, ry = res.x[:, 0], res.x[:, 1]
99     lon = np.arctan2(ry, rx) / DEG
100     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
101     ax = axes[0]
102     ax.plot(lon, res.h / 1e3, lw=1.5, color="tab:blue")
103     # 阶段着色 (用散点区分)
104     for name, color in [("vertical", "tab:blue"), ("pitch-over", "tab:green"),
105                        ("coast", "tab:orange"), ("stage2", "tab:red")]:
106         mask = np.array([p == name for p in res.phase])
107         if mask.any():
108             ax.plot(lon[mask], res.h[mask] / 1e3, lw=1.8, color=color, label=name)
109     ax.set_xlabel("经度 (deg)")
110     ax.set_ylabel("高度 h (km)")
111     ax.set_title("发射轨迹 (经度-高度)")
112     ax.legend(fontsize=8)
113     ax.grid(alpha=0.4)
114
115     ax = axes[1]
116     theta = np.linspace(0, 2 * np.pi, 400)
117     ax.plot(R_E * np.cos(theta) / 1e3, R_E * np.sin(theta) / 1e3,
118            lw=1.0, color="gray", label="地球表面")
119     ax.plot([0, 0], [-R_E / 1e3, R_E / 1e3], color="k", lw=0.5)
120     ax.plot(rx / 1e3, ry / 1e3, lw=1.8, color="tab:red", label="火箭轨迹")
121     ax.plot(rx[0] / 1e3, ry[0] / 1e3, "ko", ms=5, label="发射点")
122     ax.plot(rx[-1] / 1e3, ry[-1] / 1e3, "k*", ms=12, label="关机点")
123     ax.set_aspect("equal")
124     ax.set_xlabel("x (km)")
125     ax.set_ylabel("y (km)")
126     ax.set_title("赤道平面内轨迹")
127     ax.legend(fontsize=8, loc="lower right")
128     fig.tight_layout()
129     fig.savefig(FIG_DIR / fname, dpi=150)
130     plt.close(fig)
131
132
133 def main() -> None:
134     rk = RocketParams()
135     sim = Simulator(rk)
136     res = sim.simulate(t_coast=rk.t_coast_q1, controller=None, t_shut=None)
137
138     print("=" * 72)
139     print("问题 1: 基准控制策略仿真结果")
140     print("=" * 72)
141     print(f"一子级质量流量          mdot1 = {rk.mdot1:10.2f} kg/s")
142     print(f"一子级推进剂耗尽时刻      t1      = {rk.t_burn1:10.2f} s")
143     print(f"二子级质量流量          mdot2 = {rk.mdot2:10.2f} kg/s")
144     print(f"二子级推进剂耗尽时长      tb2     = {rk.t_burn2:10.2f} s")
145     print(f"分离时一子级关机质量        = {rk.M0 - rk.mp1:10.0f} kg")

```

```

146 print(f"二子级点火质量（分离后）      = {rk.m_after_sep:10.0f} kg")
147 print("-" * 72)
148 print("各阶段边界时刻：")
149 for name, tb in phase_boundaries(res).items():
150     print(f"    {name:<8s} t = {tb:8.2f} s" if tb is not None else f"    {name:<8s}
↪  --")
151 print("-" * 72)
152 fs = res.final_state()
153 print("二子级关机时刻状态（最终）：")
154 print(f"    轨道高度 h      = {fs['h_km']:10.2f} km    (目标 400 km)")
155 print(f"    惯性速度 V      = {fs['v_in_mps']:10.2f} m/s  (圆轨道 {rk.v_circular
↪  :.2f} m/s)")
156 print(f"    飞行路径角 gamma   = {fs['gamma_deg']:10.4f} deg (惯性) / "
157         f"{fs['gamma_rel_deg']:.4f} deg (相对大气)")
158 print(f"    总质量 m          = {fs['m_kg']:10.0f} kg    (结构+载荷 = {rk.ms2 + rk.
↪  m_payload:.0f} kg)")
159 res_orb = orbit_residual(res.x[-1], rk)
160 print(f"    入轨条件残差    = {res_orb}  (0 为精确入轨)")
161 print("-" * 72)
162 print("说明：基准策略下二子级燃料耗尽即关机，入轨条件不一定满足；")
163 print("        超出/不足的差异即问题 2 需要修正的内容。")
164
165 # ---- 保存结果 ----
166 df = pd.DataFrame({
167     "t_s": res.t,
168     "h_km": res.h / 1e3,
169     "V_inertial_mps": res.v_in,
170     "gamma_inertial_deg": res.gamma / DEG,
171     "gamma_rel_deg": res.gamma_rel / DEG,
172     "m_kg": res.m,
173     "T_N": res.T,
174     "phase": res.phase,
175 })
176 df.to_csv(RES_DIR / "q1_full_trajectory.csv", index=False)
177
178 summary = {
179     "t1_burn1_s": rk.t_burn1,
180     "t2_ignite_s": res.t2,
181     "t_shut_s": res.t_shut,
182     "h_final_km": fs["h_km"],
183     "v_final_mps": fs["v_in_mps"],
184     "gamma_final_deg": fs["gamma_deg"],
185     "gamma_rel_final_deg": fs["gamma_rel_deg"],
186     "m_final_kg": fs["m_kg"],
187     "h_target_km": rk.H_target / 1e3,
188     "v_circular_mps": rk.v_circular,
189 }
190 pd.DataFrame([summary]).to_csv(RES_DIR / "q1_summary.csv", index=False)

```

```

191     plot_profiles(res, rk, "q1_flight_profiles.png")
192     plot_trajectory(res, "q1_trajectory.png")
193     print(f"已保存: {RES_DIR / 'q1_full_trajectory.csv'}, {RES_DIR / 'q1_summary."
194           ↪ csv'}")
195     print(f"已保存: {FIG_DIR / 'q1_flight_profiles.png'}, {FIG_DIR / 'q1_trajectory"
196           ↪ .png'}")
197
198 if __name__ == "__main__":
199     main()

```

D 问题二入轨条件有界三维打靶 q2_inscription.py

```

1  """问题 2：入轨条件反演 —— 调整滑行时间与二子级俯仰角变化率（可提前关机）。
2
3  控制策略（题目给定）：
4  - 二子级推力俯仰角以恒定速率  $k$  变化，初始俯仰角与（相对大气）速度方向一致；
5  - 二子级维持额定全推力；
6  - 关机时刻  $t_{\text{shut}}$  由入轨条件决定（可提前关机）；
7  - 目标：进入 400 km 近地圆轨道。
8
9  入轨条件（惯性系，3 个等式）：
10      $|r| = R_E + H, \quad |v| = \sqrt{\mu/|r|}, \quad r \cdot v = 0$ 
11
12  未知量 ( $t_{\text{coast}}, k, t_b$ ) 共 3 个、方程 3 个 -> 有界三维打靶 (least_squares 多初
13     ↪ 值)。
14  点火前轨迹只与  $t_{\text{coast}}$  有关，缓存之；二级段单独快速积分。
15  """
16  from __future__ import annotations
17
18  import matplotlib
19
20  matplotlib.use("Agg")
21  import matplotlib.pyplot as plt
22  import numpy as np
23  import pandas as pd
24  from scipy.integrate import solve_ivp
25
26  from common import (
27      DEG,
28      GO,
29      MU,
30      RES_DIR,
31      FIG_DIR,

```



```

32     RocketParams,
33     Simulator,
34     orbit_residual,
35     thrust_unit,
36 )
37
38 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial"]
39 plt.rcParams["axes.unicode_minus"] = False
40
41
42 class InscriptionSolver:
43     """问题 2 三维打靶求解器。"""
44
45     def __init__(self, rk: RocketParams | None = None, rtol: float = 1e-9):
46         self.rk = rk or RocketParams()
47         self.rtol = rtol
48         self._ign_cache: dict[float, tuple[np.ndarray, float]] = {}
49
50     # -- 点火前状态缓存（只与 t_coast 有关）-----
51     def ignition_state(self, t_coast: float) -> tuple[np.ndarray, float]:
52         # 连续优化变量不能取整作为缓存键，否则有限差分扰动会被抹掉。
53         key = float(t_coast)
54         if key not in self._ign_cache:
55             sim = Simulator(self.rk)
56             res = sim.simulate(
57                 t_coast=key, controller=None,
58                 t_shut=self.rk.t_burn1 + key + 1e-6,
59                 rtol=self.rtol,
60             )
61             seg = res.stages[-1]
62             self._ign_cache[key] = (seg.x[0].copy(), float(seg.t[0]))
63         return self._ign_cache[key]
64
65     # -- 二级段单次积分 -----
66     def stage2_end(
67         self, x0: np.ndarray, t2: float, phi0: float,
68         k_deg_s: float, t_shut: float,
69     ) -> np.ndarray:
70         k = k_deg_s * DEG
71         rk = self.rk
72
73         def rhs(t, x):
74             p = phi0 + k * (t - t2)
75             tx_h, ty_h = thrust_unit(p, x[0], x[1])
76             T = rk.Fmax2
77             r = np.hypot(x[0], x[1])
78             ax = -MU / r**3 * x[0] + T / x[4] * tx_h
79             ay = -MU / r**3 * x[1] + T / x[4] * ty_h

```

```

80         return np.array([x[2], x[3], ax, ay, -T / (rk.Isp2 * G0)])
81
82     sol = solve_ivp(
83         rhs, (t2, t_shut), x0, method="DOP853",
84         rtol=self.rtol, atol=[1e-2, 1e-2, 1e-4, 1e-4, 1e-2],
85     )
86     return sol.y[:, -1]
87
88 # -- 残差（参数化 [t_c, k, t_b]，与论文一致）-----
89 def residual(self, z: np.ndarray) -> np.ndarray:
90     t_coast, k_deg_s, t_b = z
91     x0, t2 = self.ignition_state(t_coast)
92     phi0 = np.arctan2(x0[2] * x0[0] + x0[3] * x0[1],
93                      x0[2] * (-x0[1]) + x0[3] * x0[0])
94     t_shut = t2 + t_b
95     if t_b <= 0.0 or t_b > self.rk.t_burn2: # 超出物理边界 -> 罚
96         return np.array([1.0, 1.0, 0.0])
97     xf = self.stage2_end(x0, t2, phi0, k_deg_s, t_shut)
98     return orbit_residual(xf, self.rk)
99
100 # -- 多初值打靶（有界 least_squares）-----
101 def solve(self, starts=None) -> list[tuple[np.ndarray, float]]:
102     from scipy.optimize import least_squares
103     if starts is None:
104         starts = []
105         for tc in [0.0, 30.0, 60.0, 90.0, 120.0, 150.0, 180.0]:
106             for k in [-0.4, -0.2, -0.1, -0.05, 0.0, 0.1, 0.2]:
107                 for tb in [200.0, 300.0, 400.0, 425.0]: # 燃烧时间
108                     starts.append((tc, k, tb))
109     sols = {}
110     for z0 in starts:
111         x0, t2 = self.ignition_state(z0[0])
112         if z0[2] <= 0.0 or z0[2] > self.rk.t_burn2:
113             continue
114         try:
115             sol = least_squares(
116                 self.residual, np.array(z0, float),
117                 bounds=([0.0, -1.0, 1.0], [400.0, 1.0, self.rk.t_burn2 - 1e-6])
118                 ↪ ,
119                 xtol=1e-12, ftol=1e-12, gtol=1e-12, max_nfev=500,
120             )
121             if sol.cost < 1e-20: # 残差平方和极小 -> 高精度可行根
122                 n = float(np.linalg.norm(self.residual(sol.x)))
123                 key = (round(sol.x[0], 1), round(sol.x[1], 4))
124                 if key not in sols or n < sols[key][1]:
125                     sols[key] = (sol.x, n)
126         except Exception:
127             pass

```

```

127         return sorted(sols.values(), key=lambda kv: kv[1])
128
129
130 def main() -> None:
131     rk = RocketParams()
132     solver = InscriptionSolver(rk)
133
134     print("=" * 72)
135     print("问题 2: 入轨打靶 (网格多初值 + 有界 least_squares)")
136     print("=" * 72)
137     sols = solver.solve()
138     print(f"找到 {len(sols)} 组收敛根 (按残差排序): ")
139     for i, (sol, n) in enumerate(sols[:6]):
140         print(f"    #{i+1}: t_c={sol[0]:8.3f} s   k={sol[1]:8.4f} deg/s   "
141               f"t_b={sol[2]:8.3f} s   |res|={n:.2e}")
142
143     if not sols:
144         print("未找到可行解!")
145         return
146
147     sol, n = sols[0]
148     t_coast, k_deg_s, t_b = sol
149     t_shut = solver.ignition_state(t_coast)[1] + t_b
150
151     # ---- 完整重仿真验证 (用 Simulator 的完整流程, 含事件检测与燃尽保护) ----
152     x0, t2 = solver.ignition_state(t_coast)
153     phi0 = np.arctan2(x0[2] * x0[0] + x0[3] * x0[1],
154                      x0[2] * (-x0[1]) + x0[3] * x0[0])
155     k = k_deg_s * DEG
156
157     def controller(t):
158         return phi0 + k * (t - t2)
159
160     sim = Simulator(rk)
161     res = sim.simulate(t_coast=t_coast, controller=controller, t_shut=t_shut)
162     fs_ = res.final_state()
163
164     print("-" * 72)
165     print("最终验证 (入轨时刻状态): ")
166     print(f"    滑行时间      t_c      = {t_coast:.3f} s")
167     print(f"    俯仰角速率    k        = {k_deg_s:.4f} deg/s")
168     print(f"    燃烧时间      t_b      = {t_b:.3f} s")
169     print(f"    初始俯仰角    phi0     = {phi0 / DEG:.4f} deg (与点火时刻速度方向一致)")
170     print(f"    一子级关机    t1       = {res.t1:.3f} s; 二子级点火 t2 = {t2:.3f} s")
171     print(f"    关机时刻      t_shut   = {t_shut:.3f} s (燃尽上限 {t2 + rk.t_burn2:.3f} s"
172           ↪ ")")
173     print(f"    轨道高度      h        = {fs_['h_km']:.4f} km (目标 400)")
174     print(f"    惯性速度      V        = {fs_['v_in_mps']:.3f} m/s (目标 {rk.v_circular

```

```

↪ :.3f})")
174 print(f" 飞行路径角  gamma      = {fs_['gamma_deg']:.5f} deg (目标 0)")
175 print(f" 关机质量    m          = {fs_['m_kg']:.1f} kg (剩余推进剂 "
176       f"{fs_['m_kg'] - rk.ms2 - rk.m_payload:.1f} kg)")
177 print(f" 入轨残差      = {orbit_residual(res.x[-1], rk)}")
178
179 # ---- 保存 & 绘图 ----
180 df = pd.DataFrame({
181     "t_s": res.t, "h_km": res.h / 1e3, "V_mps": res.v_in,
182     "gamma_deg": res.gamma / DEG, "m_kg": res.m, "phase": res.phase,
183 })
184 df.to_csv(RES_DIR / "q2_trajectory.csv", index=False)
185 pd.DataFrame([
186     "t_coast_s": t_coast, "k_deg_s": k_deg_s, "t_shut_s": t_shut,
187     "phi0_deg": phi0 / DEG,
188     "h_final_km": fs_["h_km"], "v_final_mps": fs_["v_in_mps"],
189     "gamma_final_deg": fs_["gamma_deg"],
190     "m_final_kg": fs_["m_kg"],
191     "propellant_remaining_kg": fs_["m_kg"] - rk.ms2 - rk.m_payload,
192     "residual_norm": n,
193 ]).to_csv(RES_DIR / "q2_summary.csv", index=False)
194
195 fig, axes = plt.subplots(3, 1, figsize=(10, 9))
196 ax = axes[0]
197 ax.plot(res.t / 60, res.h / 1e3, lw=1.6, color="tab:blue")
198 ax.axhline(400, ls="--", color="gray", label="目标轨道 400 km")
199 ax.axvline(res.t2 / 60, ls=":", color="k", alpha=0.4, label="二级点火")
200 ax.axvline(res.t_shut / 60, ls=":", color="r", alpha=0.5, label="关机")
201 ax.set_xlabel("时间 t (min)"); ax.set_ylabel("高度 h (km)")
202 ax.set_title("问题 2 入轨轨迹: h(t)"); ax.legend(fontsize=8); ax.grid(alpha
↪ =0.4)
203
204 ax = axes[1]
205 ax.plot(res.t / 60, res.v_in / 1e3, lw=1.6, color="tab:red")
206 ax.axhline(rk.v_circular / 1e3, ls="--", color="gray",
207            label=f"圆轨道速度 {rk.v_circular/1e3:.2f} km/s")
208 ax.set_xlabel("时间 t (min)"); ax.set_ylabel("惯性速度 V (km/s)")
209 ax.set_title("V(t)"); ax.legend(fontsize=8); ax.grid(alpha=0.4)
210
211 ax = axes[2]
212 ax.plot(res.t / 60, res.gamma / DEG, lw=1.6, color="tab:green")
213 ax.axhline(0, ls="--", color="gray", label="gamma = 0")
214 ax.set_xlabel("时间 t (min)"); ax.set_ylabel("飞行路径角 gamma (deg)")
215 ax.set_title("gamma(t)"); ax.legend(fontsize=8); ax.grid(alpha=0.4)
216 fig.tight_layout()
217 fig.savefig(FIG_DIR / "q2_orbit_inscription.png", dpi=150)
218 plt.close(fig)
219 print(f"已保存: {RES_DIR / 'q2_trajectory.csv'}, {RES_DIR / 'q2_summary.csv'},

```

```

220         ↵ "
221             f"{FIG_DIR / 'q2_orbit_inscription.png'}")
222
223 if __name__ == "__main__":
224     main()

```

E 问题三、四直接配点求解器 q34_direct_collocation.py

```

1  """Questions 3 and 4 solved by Hermite--Simpson direct collocation.
2
3  The nonlinear program exposes states, controls, coast duration, and burn duration
4  as decision variables. DOP853 is used only after optimization to independently
5  re-integrate the resulting control law.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 from dataclasses import dataclass
12
13 import casadi as ca
14 import numpy as np
15 import pandas as pd
16
17 from common import DEG, GO, MU, RES_DIR, RocketParams, Simulator, orbit_residual,
18     ↵ thrust_unit
19
20 SIGMA_MIN = 0.6
21
22
23 def inertial_flight_path_angle_numeric(state: np.ndarray) -> float:
24     """Inertial flight-path angle measured from the local prograde tangent."""
25     x, y, vx, vy = np.asarray(state, dtype=float)[:4]
26     return float(np.arctan2(vx * x + vy * y, vx * (-y) + vy * x))
27
28
29 def hermite_simpson_defect_numeric(x0, x1, f0, f1, fmid, step):
30     """Return the Hermite--Simpson interval defect for numerical checks."""
31     return np.asarray(x1) - np.asarray(x0) - step * (
32         np.asarray(f0) + 4.0 * np.asarray(fmid) + np.asarray(f1)
33     ) / 6.0
34
35
36 def linear_control_integral(nodes: np.ndarray, duration: float) -> float:

```

```

37     """Integrate equally spaced, linearly interpolated control nodes."""
38     values = np.asarray(nodes, dtype=float)
39     if values.size < 2:
40         return float(values[0] * duration)
41     return float(np.trapezoid(values, dx=duration / (values.size - 1)))
42
43
44 @dataclass
45 class CollocationSolution:
46     question: int
47     n_coast: int
48     n_burn: int
49     coast_duration: float
50     burn_duration: float
51     coast_states: np.ndarray
52     burn_states: np.ndarray
53     phi_deg: np.ndarray
54     sigma: np.ndarray
55     propellant_used: float
56     objective: float
57     max_defect: float
58
59
60 class DirectCollocationSolver:
61     """Nondimensional Hermite--Simpson transcription for Q3 or Q4."""
62
63     def __init__(self, question: int, n_coast: int = 10, n_burn: int = 20):
64         if question not in (3, 4):
65             raise ValueError("question must be 3 or 4")
66         self.question = question
67         self.n_coast = n_coast
68         self.n_burn = n_burn
69         self.rk = RocketParams()
70         self.r_scale = self.rk.a_target
71         self.v_scale = self.rk.v_circular
72         self.m_scale = self.rk.m_after_sep
73
74     def scale_state(self, x: np.ndarray) -> np.ndarray:
75         return np.array([
76             x[0] / self.r_scale,
77             x[1] / self.r_scale,
78             x[2] / self.v_scale,
79             x[3] / self.v_scale,
80             x[4] / self.m_scale,
81         ])
82
83     def unscale_states(self, x: np.ndarray) -> np.ndarray:
84         scales = np.array([self.r_scale, self.r_scale, self.v_scale,

```

```

85         self.v_scale, self.m_scale)][:, None]
86     return np.asarray(x) * scales
87
88     def _dynamics(self, x, phi, sigma):
89         rx, ry, vx, vy, mass = ca.vertsplitt(x)
90         radius = ca.sqrt(rx * rx + ry * ry)
91         tx = ca.cos(phi) * (-ry / radius) + ca.sin(phi) * (rx / radius)
92         ty = ca.cos(phi) * (rx / radius) + ca.sin(phi) * (ry / radius)
93
94         gravity_scale = MU / (self.r_scale**2 * self.v_scale)
95         thrust_scale = self.rk.Fmax2 / (self.m_scale * self.v_scale)
96         return ca.vertcat(
97             self.v_scale / self.r_scale * vx,
98             self.v_scale / self.r_scale * vy,
99             -gravity_scale * rx / radius**3 + thrust_scale * sigma * tx / mass,
100            -gravity_scale * ry / radius**3 + thrust_scale * sigma * ty / mass,
101            -sigma * self.rk.Fmax2 / (self.rk.Isp2 * G0 * self.m_scale),
102        )
103
104     def _coast_dynamics(self, x):
105         rx, ry, vx, vy, _mass = ca.vertsplitt(x)
106         radius = ca.sqrt(rx * rx + ry * ry)
107         gravity_scale = MU / (self.r_scale**2 * self.v_scale)
108         return ca.vertcat(
109             self.v_scale / self.r_scale * vx,
110             self.v_scale / self.r_scale * vy,
111             -gravity_scale * rx / radius**3,
112             -gravity_scale * ry / radius**3,
113             0.0,
114        )
115
116     @staticmethod
117     def _hs_constraint(opti, x0, x1, f0, f1, fmid_fn, step):
118         xmid = 0.5 * (x0 + x1) + step * (f0 - f1) / 8.0
119         fmid = fmid_fn(xmid)
120         opti.subject_to(x1 - x0 - step * (f0 + 4.0 * fmid + f1) / 6.0 == 0)
121
122     def _reference_trajectory(self):
123         q2 = pd.read_csv(RES_DIR / "q2_summary.csv").iloc[0]
124         tc = float(q2["t_coast_s"])
125         shut = float(q2["t_shut_s"])
126         k = float(q2["k_deg_s"]) * DEG
127
128         probe = Simulator(self.rk).simulate(
129             t_coast=tc, controller=None,
130             t_shut=self.rk.t_burn1 + tc + 1e-6,
131         )
132         ignition = probe.stages[-1].x[0]

```

```

133     t2 = self.rk.t_burn1 + tc
134     phi0 = np.arctan2(
135         ignition[2] * ignition[0] + ignition[3] * ignition[1],
136         ignition[2] * (-ignition[1]) + ignition[3] * ignition[0],
137     )
138
139     def controller(t):
140         return phi0 + k * (t - t2)
141
142     result = Simulator(self.rk).simulate(
143         t_coast=tc, controller=controller, t_shut=shut, rtol=1e-10,
144     )
145     return tc, shut - t2, phi0, k, result
146
147     @staticmethod
148     def _sample_state(result, query_times):
149         return np.vstack([
150             np.interp(query_times, result.t, result.x[:, j])
151             for j in range(result.x.shape[1])
152         ])
153
154     def solve(self, warm: CollocationSolution | None = None, print_level: int = 0):
155         opti = ca.Opti()
156         nc, nb = self.n_coast, self.n_burn
157         tc = opti.variable()
158         tb = opti.variable()
159         xc = opti.variable(5, nc + 1)
160         xb = opti.variable(5, nb + 1)
161         phi = opti.variable(1, nb + 1)
162         sigma = opti.variable(1, nb + 1) if self.question == 4 else ca.DM.ones(1,
163 ↪ nb + 1)
164
165         # Fixed state immediately after first-stage separation.
166         baseline = Simulator(self.rk).simulate(
167             t_coast=0.0, controller=None,
168             t_shut=self.rk.t_burn1 + 1e-6, rtol=1e-10,
169         )
170         x_sep = self.scale_state(baseline.stages[-1].x[0])
171         opti.subject_to(xc[:, 0] == x_sep)
172         opti.subject_to(xb[:, 0] == xc[:, -1])
173
174         opti.subject_to(opti.bounded(0.0, tc, 400.0))
175         burn_upper = self.rk.t_burn2 if self.question == 3 else self.rk.t_burn2 /
176 ↪ SIGMA_MIN
177         opti.subject_to(opti.bounded(1.0, tb, burn_upper))
178         opti.subject_to(opti.bounded(-0.5 * np.pi, phi, 0.5 * np.pi))
179         opti.subject_to(xc[4, :] == 1.0)
180         opti.subject_to(opti.bounded(

```



```

179         (self.rk.ms2 + self.rk.m_payload) / self.m_scale,
180         xb[4, :], 1.0,
181     ))
182     if self.question == 4:
183         opti.subject_to(opti.bounded(SIGMA_MIN, sigma, 1.0))
184
185     hc = tc / nc
186     for k_idx in range(nc):
187         x0, x1 = xc[:, k_idx], xc[:, k_idx + 1]
188         f0, f1 = self._coast_dynamics(x0), self._coast_dynamics(x1)
189         self._hs_constraint(opti, x0, x1, f0, f1,
190                             self._coast_dynamics, hc)
191
192     hb = tb / nb
193     for k_idx in range(nb):
194         x0, x1 = xb[:, k_idx], xb[:, k_idx + 1]
195         p0, p1 = phi[0, k_idx], phi[0, k_idx + 1]
196         s0, s1 = sigma[0, k_idx], sigma[0, k_idx + 1]
197         f0 = self._dynamics(x0, p0, s0)
198         f1 = self._dynamics(x1, p1, s1)
199         pmid, smid = 0.5 * (p0 + p1), 0.5 * (s0 + s1)
200         self._hs_constraint(
201             opti, x0, x1, f0, f1,
202             lambda xmid, p=pmid, s=smid: self._dynamics(xmid, p, s), hb,
203         )
204
205     rf = xb[0:2, -1]
206     vf = xb[2:4, -1]
207     opti.subject_to(ca.sumsq(rf) == 1.0)
208     opti.subject_to(ca.dot(rf, vf) == 0.0)
209     opti.subject_to(rf[0] * vf[1] - rf[1] * vf[0] == 1.0)
210
211     propellant_used = (1.0 - xb[4, -1]) * self.m_scale
212     opti.minimize(propellant_used)
213
214     tc0, tb0, phi0, k0, reference = self._reference_trajectory()
215     if warm is None:
216         coast_t = np.linspace(self.rk.t_burn1, self.rk.t_burn1 + tc0, nc + 1)
217         burn_t = np.linspace(self.rk.t_burn1 + tc0,
218                             self.rk.t_burn1 + tc0 + tb0, nb + 1)
219         opti.set_initial(tc, tc0)
220         opti.set_initial(tb, tb0)
221         opti.set_initial(xc, self.scale_state(self._sample_state(reference,
222 ↪ coast_t)))
223         opti.set_initial(xb, self.scale_state(self._sample_state(reference,
224 ↪ burn_t)))
225         opti.set_initial(phi, phi0 + k0 * (burn_t - burn_t[0]))
226         if self.question == 4:

```

```

225         opti.set_initial(sigma, 1.0)
226     else:
227         opti.set_initial(tc, warm.coast_duration)
228         opti.set_initial(tb, warm.burn_duration)
229         old_c = np.linspace(0.0, 1.0, warm.coast_states.shape[1])
230         old_b = np.linspace(0.0, 1.0, warm.burn_states.shape[1])
231         new_c = np.linspace(0.0, 1.0, nc + 1)
232         new_b = np.linspace(0.0, 1.0, nb + 1)
233         xc_guess = np.vstack([np.interp(new_c, old_c, row) for row in warm.
↪ coast_states])
234         xb_guess = np.vstack([np.interp(new_b, old_b, row) for row in warm.
↪ burn_states])
235         opti.set_initial(xc, xc_guess)
236         opti.set_initial(xb, xb_guess)
237         opti.set_initial(phi, np.interp(new_b, old_b, warm.phi_deg) * DEG)
238         if self.question == 4:
239             opti.set_initial(sigma, np.interp(new_b, old_b, warm.sigma))
240
241     options = {
242         "expand": True,
243         "ipopt.print_level": print_level,
244         "ipopt.max_iter": 1200,
245         "ipopt.tol": 1e-8,
246         "ipopt.acceptable_tol": 1e-6,
247         "print_time": bool(print_level),
248     }
249     opti.solver("ipopt", options)
250     solved = opti.solve()
251
252     xc_value = np.asarray(solved.value(xc))
253     xb_value = np.asarray(solved.value(xb))
254     phi_value = np.asarray(solved.value(phi)).reshape(-1) / DEG
255     sigma_value = (np.asarray(solved.value(sigma)).reshape(-1)
256                    if self.question == 4 else np.ones(nb + 1))
257     stats = solved.stats()
258     return CollocationSolution(
259         question=self.question,
260         n_coast=nc,
261         n_burn=nb,
262         coast_duration=float(solved.value(tc)),
263         burn_duration=float(solved.value(tb)),
264         coast_states=xc_value,
265         burn_states=xb_value,
266         phi_deg=phi_value,
267         sigma=sigma_value,
268         propellant_used=float(solved.value(propellant_used)),
269         objective=float(solved.value(propellant_used)),
270         max_defect=float(stats.get("iterations", {}).get("inf_pr", [np.nan]))

```

```

↪ [-1]),
271     )
272
273     def validate(self, solution: CollocationSolution, rtol: float = 1e-11):
274         t2 = self.rk.t_burn1 + solution.coast_duration
275         t3 = t2 + solution.burn_duration
276         tau_nodes = np.linspace(0.0, 1.0, solution.n_burn + 1)
277
278         def controller(t):
279             tau = np.clip((t - t2) / solution.burn_duration, 0.0, 1.0)
280             return np.interp(tau, tau_nodes, solution.phi_deg) * DEG
281
282         def sigma_fn(t):
283             tau = np.clip((t - t2) / solution.burn_duration, 0.0, 1.0)
284             return float(np.interp(tau, tau_nodes, solution.sigma))
285
286         result = Simulator(self.rk).simulate(
287             t_coast=solution.coast_duration,
288             controller=controller,
289             sigma=sigma_fn,
290             t_shut=t3,
291             rtol=rtol,
292         )
293         final = result.x[-1]
294         radius = np.hypot(final[0], final[1])
295         vr = (final[0] * final[2] + final[1] * final[3]) / radius
296         vt = (-final[1] * final[2] + final[0] * final[3]) / radius
297         return result, {
298             "height_error_m": radius - self.rk.a_target,
299             "radial_velocity_error_mps": vr,
300             "tangential_velocity_error_mps": vt - self.rk.v_circular,
301             "residual_norm": float(np.linalg.norm(orbit_residual(final, self.rk))),
302             "propellant_used_kg": self.rk.m_after_sep - final[4],
303         }
304
305
306     def save_summary(solution: CollocationSolution, validation: dict):
307         # 控制节点以完整精度（17 位有效数字）保存，保证公开数据可精确复现独立重积分。
308         row = {
309             "question": solution.question,
310             "n_coast": solution.n_coast,
311             "n_burn": solution.n_burn,
312             "t_coast_s": solution.coast_duration,
313             "burn_duration_s": solution.burn_duration,
314             "t_shut_s": RocketParams().t_burn1 + solution.coast_duration + solution.
↪ burn_duration,
315             "propellant_used_collocation_kg": solution.propellant_used,
316             "propellant_used_reintegration_kg": validation["propellant_used_kg"],

```

```

317     "height_error_m": validation["height_error_m"],
318     "radial_velocity_error_mps": validation["radial_velocity_error_mps"],
319     "tangential_velocity_error_mps": validation["tangential_velocity_error_mps"]
↪ ],
320     "residual_norm": validation["residual_norm"],
321     "phi_nodes_deg": " | ".join(f"{v:.17g}" for v in solution.phi_deg),
322     "sigma_nodes": " | ".join(f"{v:.17g}" for v in solution.sigma),
323 }
324 path = RES_DIR / f"q{solution.question}_collocation_summary.csv"
325 pd.DataFrame([row]).to_csv(path, index=False)
326 return path
327
328
329 def main():
330     parser = argparse.ArgumentParser()
331     parser.add_argument("--question", type=int, choices=(3, 4), required=True)
332     parser.add_argument("--n-coast", type=int, default=10)
333     parser.add_argument("--n-burn", type=int, default=20)
334     parser.add_argument("--print-level", type=int, default=0)
335     args = parser.parse_args()
336
337     solver = DirectCollocationSolver(args.question, args.n_coast, args.n_burn)
338     solution = solver.solve(print_level=args.print_level)
339     _trajectory, validation = solver.validate(solution)
340     path = save_summary(solution, validation)
341     print(f"Q{args.question} Hermite-Simpson: coast={solution.coast_duration:.6f} s
↪ , "
342         f"burn={solution.burn_duration:.6f} s, fuel={solution.propellant_used:.3f
↪ } kg")
343     print("DOP853 reintegration:", validation)
344     print("saved:", path)
345
346
347 if __name__ == "__main__":
348     main()

```

F 一键复现脚本 run_all.py

```

1  """单一公开入口：完整复现论文全部数值结果与图表。
2
3  用法（仓库根目录）：
4      python src/run_all.py
5
6  执行内容：
7      Q1 基准策略仿真 -> results/q1_*.csv, figures/q1_*.png
8      Q2 有界三维打靶 -> results/q2_*.csv

```

```

9   Q3 配点 10/20 与 20/40 (细网格解插值重解) -> results/q3_collocation_summary.csv
10  Q4 配点 20/40 (满推力与低节流两类初值) -> results/q4_collocation_summary.csv
11  图表 fig_q3_trajectory/controls, fig_q4_trajectory/controls
12  """
13  from __future__ import annotations
14
15  import subprocess
16  import sys
17  from pathlib import Path
18
19  ROOT = Path(__file__).resolve().parent.parent
20
21
22  def run(cmd: list[str], cwd: Path | None = None) -> None:
23      print(">>>", " ".join(cmd))
24      subprocess.run(cmd, cwd=cwd or ROOT, check=True)
25
26
27  def main() -> None:
28      py = sys.executable
29
30      # Q1
31      run([py, "src/q1_baseline.py"])
32      # Q2
33      run([py, "src/q2_inscription.py"])
34      # Q3 粗网格 -> 细网格
35      run([py, "src/q34_direct_collocation.py",
36           "--question", "3", "--n-coast", "10", "--n-burn", "20"])
37      run([py, "src/q34_direct_collocation.py",
38           "--question", "3", "--n-coast", "20", "--n-burn", "40"])
39      # Q4 细网格 (两类初值由求解器内部完成)
40      run([py, "src/q34_direct_collocation.py",
41           "--question", "4", "--n-coast", "20", "--n-burn", "40"])
42      # 论文图表
43      run([py, str(ROOT / "figures_src" / "_gen_q34_plots.py")])
44
45      print("\n全部复现完成。数值见 results/, 图表见 论文/ 与 figures/。")
46
47
48  if __name__ == "__main__":
49      main()

```

G 交叉验证独立实现 rocket_trajectory_solution.py

```

1  """Auditable model and solution for the two-stage launch problem.
2

```

```

3 Numerical layers are kept separate: fixed-step RK4 for forward simulation,
4 a Newton predictor-corrector for Question 2, and Hermite-Simpson direct
5 collocation for the continuous optimal-control problems in Questions 3 and 4.
6 """
7
8 from __future__ import annotations
9
10 import json
11 import math
12 from dataclasses import asdict, dataclass
13 from pathlib import Path
14
15 import matplotlib.pyplot as plt
16 import numpy as np
17 import pandas as pd
18 from scipy.optimize import minimize
19
20
21 BASE_DIR = Path(__file__).resolve().parent
22 OUT_DIR = BASE_DIR.parent / "结果"
23 FIG_DIR = OUT_DIR / "figures"
24
25
26 @dataclass(frozen=True)
27 class Constants:
28     re: float = 6_371_000.0
29     mu: float = 3.986004418e14
30     omega: float = 7.2921159e-5
31     g0: float = 9.80665
32     rho0: float = 1.225
33     h0: float = 7_200.0
34     cd: float = 0.3
35     area: float = 12.5
36     target_h: float = 400_000.0
37
38
39 C = Constants()
40 R_TARGET = C.re + C.target_h
41 V_CIRC = math.sqrt(C.mu / R_TARGET)
42
43 M0, MS1, MP1, ISP1, T1 = 500_000.0, 40_000.0, 380_000.0, 300.0, 7.5e6
44 MS2, MP2, M_PAYLOAD, ISP2, T2_MAX = 8_000.0, 62_000.0, 10_000.0, 420.0, 6.0e5
45 TB1 = MP1 * ISP1 * C.g0 / T1
46 TB2_MAX = MP2 * ISP2 * C.g0 / T2_MAX
47 M2_INITIAL = MS2 + MP2 + M_PAYLOAD
48 M2_DRY_PAYLOAD = MS2 + M_PAYLOAD
49 MDOT2_MAX = T2_MAX / (ISP2 * C.g0)
50 R_SCALE, V_SCALE, M_SCALE = R_TARGET, V_CIRC, M2_INITIAL

```

```

51
52
53 def flight_path_angle(state):
54     return math.atan2(state[2], state[3])
55
56
57 def terminal_metrics(state):
58     r, _, vr, vt, mass = state
59     return {
60         "height_km": (r - C.re) / 1_000.0,
61         "inertial_speed_m_s": math.hypot(vr, vt),
62         "ground_relative_speed_m_s": math.hypot(vr, vt - C.omega * r),
63         "radial_speed_m_s": vr,
64         "tangential_speed_m_s": vt,
65         "flight_path_deg": math.degrees(math.atan2(vr, vt)),
66         "mass_kg": mass,
67     }
68
69
70 def rhs(_time, state, thrust, isp, theta, include_drag):
71     """Variable-mass point dynamics in an Earth-centered inertial frame."""
72     r, _phi, vr, vt, mass = state
73     drag_r = drag_t = 0.0
74     if include_drag:
75         height = max(0.0, r - C.re)
76         relative_t = vt - C.omega * r
77         relative_speed = math.hypot(vr, relative_t)
78         if relative_speed > 1e-12:
79             density = C.rho0 * math.exp(-height / C.h0)
80             drag = 0.5 * density * relative_speed**2 * C.cd * C.area
81             drag_r = -drag * vr / relative_speed
82             drag_t = -drag * relative_t / relative_speed
83     return np.array(
84         [
85             vr,
86             vt / r,
87             vt**2 / r - C.mu / r**2 + (thrust * math.sin(theta) + drag_r) / mass,
88             -vr * vt / r + (thrust * math.cos(theta) + drag_t) / mass,
89             -thrust / (isp * C.g0) if thrust > 0 else 0.0,
90         ]
91     )
92
93
94 def rk4_segment(state0, duration, control, include_drag, n_steps):
95     """Classical fourth-order Runge-Kutta formula on an equal time mesh."""
96     if duration <= 0:
97         return np.array([0.0]), np.array([state0], dtype=float)
98     n_steps = max(1, int(n_steps))

```

```

99     dt = duration / n_steps
100     times = np.linspace(0.0, duration, n_steps + 1)
101     states = np.empty((n_steps + 1, len(state0)))
102     states[0] = state0
103     for index in range(n_steps):
104         time, state = times[index], states[index]
105
106         def derivative(local_time, local_state):
107             thrust, isp, theta = control(local_time, local_state)
108             return rhs(local_time, local_state, thrust, isp, theta, include_drag)
109
110         k1 = derivative(time, state)
111         k2 = derivative(time + dt / 2, state + dt * k1 / 2)
112         k3 = derivative(time + dt / 2, state + dt * k2 / 2)
113         k4 = derivative(time + dt, state + dt * k3)
114         states[index + 1] = state + dt * (k1 + 2 * k2 + 2 * k3 + k4) / 6
115     return times, states
116
117
118 def stage1_pitch(time):
119     return math.pi / 2 if time <= 10 else math.pi / 2 - math.radians(0.4) * (time -
120 ↪ 10)
121
122 def simulate_stage1(step=0.25):
123     initial = np.array([C.re, 0.0, 0.0, C.omega * C.re, M0])
124
125     def control(time, _state):
126         return T1, ISP1, stage1_pitch(time)
127
128     times, states = rk4_segment(initial, TB1, control, True, math.ceil(TB1 / step))
129     states[-1, 4] = M0 - MP1
130     separated = states[-1].copy()
131     separated[4] -= MS1
132     return times, states, separated
133
134
135 STAGE1_TIME, STAGE1_STATE, STAGE1_SEPARATED = simulate_stage1()
136
137
138 def simulate_coast(state0, duration, n_steps=320):
139     return rk4_segment(state0, duration, lambda _t, _x: (0.0, ISP2, 0.0), False,
140 ↪ n_steps)
141
142 def interpolate_control(values, tau):
143     return float(np.interp(np.clip(tau, 0.0, 1.0), np.linspace(0.0, 1.0, len(values
144 ↪ )), values))

```



```

144
145
146 def simulate_stage2(state0, duration, theta_function, throttle_function=None,
    ↪ n_steps=1200):
147     throttle_function = throttle_function or (lambda _time: 1.0)
148
149     def control(time, _state):
150         throttle = float(np.clip(throttle_function(time), 0.0, 1.0))
151         return throttle * T2_MAX, ISP2, float(theta_function(time))
152
153     return rk4_segment(state0, duration, control, False, n_steps)
154
155
156 def combine_segments(segments):
157     time_parts, state_parts, offset = [], [], 0.0
158     for index, (times, states) in enumerate(segments):
159         if index:
160             times, states = times[1:], states[1:]
161             time_parts.append(times + offset)
162             state_parts.append(states)
163             offset += float(times[-1]) if len(times) else 0.0
164     return np.concatenate(time_parts), np.vstack(state_parts)
165
166
167 def q1_baseline(step=0.25):
168     t1, y1, separated = simulate_stage1(step)
169     tc, yc = simulate_coast(separated, 60.0, math.ceil(60.0 / step))
170
171     def control(_time, state):
172         return T2_MAX, ISP2, flight_path_angle(state)
173
174     tb, yb = rk4_segment(yc[-1], TB2_MAX, control, False, math.ceil(TB2_MAX / step)
    ↪ )
175     times, states = combine_segments([(t1, y1), (tc, yc), (tb, yb)])
176     return {
177         "name": "Q1 baseline",
178         "coast_time_s": 60.0,
179         "burn_time_s": TB2_MAX,
180         "stage2_prop_used_kg": MP2,
181         "time_s": times,
182         "state": states,
183         "terminal": terminal_metrics(states[-1]),
184     }
185
186
187 def terminal_residual(state):
188     return np.array([(state[0] - R_TARGET) / 1_000.0, state[2] / 10.0, (state[3] -
    ↪ V_CIRC) / 10.0])

```

```

189
190
191 def q2_terminal(decision, detailed=False):
192     coast_time, rate_deg_s, burn_time = decision
193     _, coast = simulate_coast(STAGE1_SEPARATED, coast_time, 240)
194     theta0 = flight_path_angle(coast[-1])
195     theta = lambda time: theta0 + math.radians(rate_deg_s) * time
196     burn_times, burn_states = simulate_stage2(coast[-1], burn_time, theta, n_steps
↪ =1600 if detailed else 700)
197     return burn_times, burn_states, theta0
198
199
200 def solve_q2():
201     """Newton predictor-corrector using an explicit terminal sensitivity matrix."""
202     decision = np.array([100.0, -0.05, 398.0])
203     lower, upper = np.array([0.0, -0.35, 40.0]), np.array([1_200.0, 0.20, TB2_MAX])
204     increments = np.array([0.05, 2e-5, 0.05])
205     records = []
206
207     def residual_at(value):
208         return terminal_residual(q2_terminal(value)[1][-1])
209
210     for iteration in range(20):
211         residual = residual_at(decision)
212         jacobian = np.empty((3, 3))
213         for column in range(3):
214             plus, minus = decision.copy(), decision.copy()
215             plus[column] += increments[column]
216             minus[column] -= increments[column]
217             jacobian[:, column] = (residual_at(plus) - residual_at(minus)) / (2 *
↪ increments[column])
218         correction = np.linalg.solve(jacobian, -residual)
219         records.append(
220             {
221                 "iteration": iteration,
222                 "coast_time_s": decision[0],
223                 "pitch_rate_deg_s": decision[1],
224                 "burn_time_s": decision[2],
225                 "residual_norm": np.linalg.norm(residual),
226                 "jacobian_condition": np.linalg.cond(jacobian),
227                 "scaled_correction_norm": np.linalg.norm(correction / np.array
↪ ([100.0, 0.05, 100.0])),
228             }
229         )
230         if np.linalg.norm(residual) < 1e-9:
231             break
232         for exponent in range(12):
233             candidate = np.clip(decision + (0.5**exponent) * correction, lower,

```

```

↪ upper)
234         if np.linalg.norm(residual_at(candidate)) < np.linalg.norm(residual):
235             decision = candidate
236             break
237         else:
238             raise RuntimeError("Q2 Newton corrector could not reduce the terminal
↪ residual")
239
240 _, states, theta0 = q2_terminal(decision, detailed=True)
241 return {
242     "name": "Q2 Newton targetting",
243     "coast_time_s": decision[0],
244     "rate_deg_s": decision[1],
245     "burn_time_s": decision[2],
246     "stage2_prop_used_kg": MDOT2_MAX * decision[2],
247     "theta0_rad": theta0,
248     "iterations": records,
249     "terminal": terminal_metrics(states[-1]),
250     "residual_norm": float(np.linalg.norm(terminal_residual(states[-1]))),
251 }
252
253
254 def coast_initial_scaled(coast_time):
255     _, states = simulate_coast(STAGE1_SEPARATED, coast_time, 160)
256     return states[-1, [0, 2, 3]] / np.array([R_SCALE, V_SCALE, V_SCALE])
257
258
259 def vacuum_rhs_physical(state, theta, throttle):
260     r, vr, vt, mass = state
261     thrust = throttle * T2_MAX
262     return np.array(
263         [
264             vr,
265             vt**2 / r - C.mu / r**2 + thrust * math.sin(theta) / mass,
266             -vr * vt / r + thrust * math.cos(theta) / mass,
267             -thrust / (ISP2 * C.g0),
268         ]
269     )
270
271
272 def q3_scaled_rhs(state, theta, tau, burn_time):
273     r, vr, vt = state * np.array([R_SCALE, V_SCALE, V_SCALE])
274     mass = M2_INITIAL - MDOT2_MAX * burn_time * tau
275     derivative = vacuum_rhs_physical(np.array([r, vr, vt, mass]), theta, 1.0)[:3]
276     return burn_time * derivative / np.array([R_SCALE, V_SCALE, V_SCALE])
277
278
279 def q4_scaled_rhs(state, theta, throttle, burn_time):

```

```

280     physical_state = state * np.array([R_SCALE, V_SCALE, V_SCALE, M_SCALE])
281     derivative = vacuum_rhs_physical(physical_state, theta, throttle)
282     return burn_time * derivative / np.array([R_SCALE, V_SCALE, V_SCALE, M_SCALE])
283
284
285 def hermite_simpson_q3(states, angles, burn_time):
286     count, defects = len(angles), []
287     step = 1.0 / (count - 1)
288     for index in range(count - 1):
289         tau_left, tau_right = index * step, (index + 1) * step
290         f_left = q3_scaled_rhs(states[index], angles[index], tau_left, burn_time)
291         f_right = q3_scaled_rhs(states[index + 1], angles[index + 1], tau_right,
↳ burn_time)
292         state_mid = (states[index] + states[index + 1]) / 2 + step * (f_left -
↳ f_right) / 8
293         f_mid = q3_scaled_rhs(state_mid, (angles[index] + angles[index + 1]) / 2,
↳ tau_left + step / 2, burn_time)
294         defects.append(states[index + 1] - states[index] - step * (f_left + 4 *
↳ f_mid + f_right) / 6)
295     return np.concatenate(defects)
296
297
298 def hermite_simpson_q4(states, angles, throttles, burn_time):
299     count, defects = len(angles), []
300     step = 1.0 / (count - 1)
301     for index in range(count - 1):
302         f_left = q4_scaled_rhs(states[index], angles[index], throttles[index],
↳ burn_time)
303         f_right = q4_scaled_rhs(states[index + 1], angles[index + 1], throttles[
↳ index + 1], burn_time)
304         state_mid = (states[index] + states[index + 1]) / 2 + step * (f_left -
↳ f_right) / 8
305         f_mid = q4_scaled_rhs(
306             state_mid,
307             (angles[index] + angles[index + 1]) / 2,
308             (throttles[index] + throttles[index + 1]) / 2,
309             burn_time,
310         )
311         defects.append(states[index + 1] - states[index] - step * (f_left + 4 *
↳ f_mid + f_right) / 6)
312     return np.concatenate(defects)
313
314
315 def q3_seed_from_q2(q2, nodes):
316     coast_time, burn_time = q2["coast_time_s"], q2["burn_time_s"]
317     _, coast = simulate_coast(STAGE1_SEPARATED, coast_time, 320)
318     theta0, rate = flight_path_angle(coast[-1]), math.radians(q2["rate_deg_s"])
319     theta = lambda time: theta0 + rate * time

```

```

320     _, burn = simulate_stage2(coast[-1], burn_time, theta, n_steps=nodes - 1)
321     states = burn[:, [0, 2, 3]] / np.array([R_SCALE, V_SCALE, V_SCALE])
322     angles = np.array([theta(time) for time in np.linspace(0.0, burn_time, nodes)])
323     return coast_time, burn_time, states, angles
324
325
326 def refine_q3_seed(solution, nodes):
327     old_tau, new_tau = np.linspace(0, 1, solution["nodes"]), np.linspace(0, 1,
↪ nodes)
328     states = np.column_stack(
329         [np.interp(new_tau, old_tau, solution["states_scaled"][:, column]) for
↪ column in range(3)]
330     )
331     angles = np.interp(new_tau, old_tau, solution["angles_rad"])
332     return solution["coast_time_s"], solution["burn_time_s"], states, angles
333
334
335 def solve_q3_collocation(q2, nodes, previous=None):
336     coast0, burn0, states0, angles0 = (
337         q3_seed_from_q2(q2, nodes) if previous is None else refine_q3_seed(previous
↪ , nodes)
338     )
339     initial = np.r_[coast0 / 300.0, burn0 / TB2_MAX, states0.ravel(), angles0]
340
341     def unpack(vector):
342         return (
343             vector[0] * 300.0,
344             vector[1] * TB2_MAX,
345             vector[2 : 2 + 3 * nodes].reshape(nodes, 3),
346             vector[2 + 3 * nodes :],
347         )
348
349     def equality(vector):
350         coast_time, burn_time, states, angles = unpack(vector)
351         return np.r_[
352             states[0] - coast_initial_scaled(coast_time),
353             hermite_simpson_q3(states, angles, burn_time),
354             states[-1] - np.array([1.0, 0.0, 1.0]),
355         ]
356
357     lower = np.r_[
358         0.0,
359         300.0 / TB2_MAX,
360         np.tile([0.93, -0.4, 0.0], nodes),
361         np.full(nodes, math.radians(-45.0)),
362     ]
363     upper = np.r_[4.0, 1.0, np.tile([1.25, 0.5, 1.25], nodes), np.full(nodes, math.
↪ radians(90.0))]

```

```

364     history = []
365
366     def callback(vector):
367         history.append(
368             {
369                 "iteration": len(history),
370                 "burn_time_s": vector[1] * TB2_MAX,
371                 "max_equality_residual": np.max(np.abs(equality(vector))),
372             }
373         )
374
375     result = minimize(
376         lambda vector: vector[1],
377         initial,
378         method="SLSQP",
379         bounds=list(zip(lower, upper)),
380         constraints={"type": "eq", "fun": equality},
381         callback=callback,
382         options={"ftol": 1e-10, "maxiter": 800, "disp": False},
383     )
384     if not result.success:
385         raise RuntimeError(f"Q3 collocation failed on {nodes} nodes: {result.
↪ message}")
386     coast_time, burn_time, states, angles = unpack(result.x)
387     return {
388         "nodes": nodes,
389         "coast_time_s": coast_time,
390         "burn_time_s": burn_time,
391         "stage2_prop_used_kg": MDOT2_MAX * burn_time,
392         "states_scaled": states,
393         "angles_rad": angles,
394         "max_collocation_defect": float(np.max(np.abs(equality(result.x)))),
395         "iterations": int(result.nit),
396         "history": history,
397     }
398
399
400 def verify_q3(solution, steps=10_000):
401     coast_time, burn_time, angles = solution["coast_time_s"], solution["burn_time_s
↪ "], solution["angles_rad"]
402     _, coast = simulate_coast(STAGE1_SEPARATED, coast_time, 2_000)
403     theta = lambda time: interpolate_control(angles, time / burn_time)
404     times, states = simulate_stage2(coast[-1], burn_time, theta, n_steps=steps)
405     residual = terminal_residual(states[-1])
406     return {
407         "times": times,
408         "states": states,
409         "terminal": terminal_metrics(states[-1]),

```

```

410         "scaled_terminal_residual": residual,
411         "terminal_residual_norm": float(np.linalg.norm(residual)),
412     }
413
414
415 def q4_state_angle_bounds(nodes):
416     lower = np.r_[
417         0.0,
418         300.0 / TB2_MAX,
419         np.tile([0.93, -0.4, 0.0, M2_DRY_PAYLOAD / M_SCALE], nodes),
420         np.full(nodes, math.radians(-45.0)),
421     ]
422     upper = np.r_[
423         4.0,
424         1.0 / 0.6,
425         np.tile([1.25, 0.5, 1.25, 1.0], nodes),
426         np.full(nodes, math.radians(90.0)),
427     ]
428     return lower, upper
429
430
431 def solve_q4_fixed_throttle(q3, throttle):
432     """Optimize the trajectory with one prescribed throttle level for sensitivity
433     ↪ analysis."""
434     nodes = q3["nodes"]
435     burn0 = q3["burn_time_s"] / throttle
436     tau = np.linspace(0.0, 1.0, nodes)
437     mass = (M2_INITIAL - MDOT2_MAX * throttle * burn0 * tau) / M_SCALE
438     states0 = np.column_stack([q3["states_scaled"], mass])
439     initial = np.r_[q3["coast_time_s"] / 300.0, burn0 / TB2_MAX, states0.ravel(),
440     ↪ q3["angles_rad"]]
441
442     def unpack(vector):
443         return (
444             vector[0] * 300.0,
445             vector[1] * TB2_MAX,
446             vector[2 : 2 + 4 * nodes].reshape(nodes, 4),
447             vector[2 + 4 * nodes :],
448         )
449
450     def equality(vector):
451         coast_time, burn_time, states, angles = unpack(vector)
452         throttles = np.full(nodes, throttle)
453         return np.r_[
454             states[0] - np.r_[coast_initial_scaled(coast_time), 1.0],
455             hermite_simpson_q4(states, angles, throttles, burn_time),
456             states[-1, :3] - np.array([1.0, 0.0, 1.0]),
457         ]

```

```

456
457 lower, upper = q4_state_angle_bounds(nodes)
458 result = minimize(
459     lambda vector: -unpack(vector)[2][-1, 3],
460     initial,
461     method="SLSQP",
462     bounds=list(zip(lower, upper)),
463     constraints={"type": "eq", "fun": equality},
464     options={"ftol": 1e-10, "maxiter": 1_200, "disp": False},
465 )
466 if not result.success:
467     raise RuntimeError(f"Q4 fixed-throttle solve failed at eta={throttle}: {
↵ result.message}")
468 coast_time, burn_time, states, angles = unpack(result.x)
469 return {
470     "seed_throttle": throttle,
471     "coast_time_s": coast_time,
472     "burn_time_s": burn_time,
473     "stage2_prop_used_kg": M2_INITIAL - states[-1, 3] * M_SCALE,
474     "states_scaled": states,
475     "angles_rad": angles,
476     "max_collocation_defect": float(np.max(np.abs(equality(result.x)))),
477     "iterations": int(result.nit),
478 }
479
480
481 def release_q4_throttle(fixed_solution):
482     """Release all throttle nodes from a feasible constant-throttle solution."""
483     nodes = len(fixed_solution["angles_rad"])
484     seed_throttle = fixed_solution["seed_throttle"]
485     initial = np.r_[
486         fixed_solution["coast_time_s"] / 300.0,
487         fixed_solution["burn_time_s"] / TB2_MAX,
488         fixed_solution["states_scaled"].ravel(),
489         fixed_solution["angles_rad"],
490         np.full(nodes, seed_throttle),
491     ]
492
493 def unpack(vector):
494     return (
495         vector[0] * 300.0,
496         vector[1] * TB2_MAX,
497         vector[2 : 2 + 4 * nodes].reshape(nodes, 4),
498         vector[2 + 4 * nodes : 2 + 5 * nodes],
499         vector[2 + 5 * nodes :],
500     )
501
502 def equality(vector):

```



```

503     coast_time, burn_time, states, angles, throttles = unpack(vector)
504     return np.r_[
505         states[0] - np.r_[coast_initial_scaled(coast_time), 1.0],
506         hermite_simpson_q4(states, angles, throttles, burn_time),
507         states[-1, :3] - np.array([1.0, 0.0, 1.0]),
508     ]
509
510     lower, upper = q4_state_angle_bounds(nodes)
511     lower = np.r_[lower, np.full(nodes, 0.6)]
512     upper = np.r_[upper, np.ones(nodes)]
513     result = minimize(
514         lambda vector: -unpack(vector)[2][-1, 3],
515         initial,
516         method="SLSQP",
517         bounds=list(zip(lower, upper)),
518         constraints={"type": "eq", "fun": equality},
519         options={"ftol": 1e-10, "maxiter": 1_500, "disp": False},
520     )
521     if not result.success:
522         raise RuntimeError(f"Q4 released-throttle solve failed from eta={
↵ seed_throttle}: {result.message}")
523     coast_time, burn_time, states, angles, throttles = unpack(result.x)
524     return {
525         "name": "Q4 throttle optimal control",
526         "nodes": nodes,
527         "seed_throttle": seed_throttle,
528         "coast_time_s": coast_time,
529         "burn_time_s": burn_time,
530         "stage2_prop_used_kg": M2_INITIAL - states[-1, 3] * M_SCALE,
531         "states_scaled": states,
532         "angles_rad": angles,
533         "throttles": throttles,
534         "max_collocation_defect": float(np.max(np.abs(equality(result.x)))),
535         "iterations": int(result.nit),
536     }
537
538
539 def solve_q4_collocation(q3):
540     fixed_solutions = [solve_q4_fixed_throttle(q3, throttle) for throttle in [1.0,
↵ 0.9, 0.8, 0.7, 0.6]]
541     candidates = [release_q4_throttle(solution) for solution in fixed_solutions]
542     best = min(candidates, key=lambda solution: solution["stage2_prop_used_kg"])
543     best["constant_throttle_audit"] = [
544         {
545             "throttle": solution["seed_throttle"],
546             "coast_time_s": solution["coast_time_s"],
547             "burn_time_s": solution["burn_time_s"],
548             "stage2_prop_used_kg": solution["stage2_prop_used_kg"],

```

```

549         "max_collocation_defect": solution["max_collocation_defect"],
550         "iterations": solution["iterations"],
551     }
552     for solution in fixed_solutions
553 ]
554 best["multistart_audit"] = [
555     {
556         "seed_throttle": solution["seed_throttle"],
557         "coast_time_s": solution["coast_time_s"],
558         "burn_time_s": solution["burn_time_s"],
559         "stage2_prop_used_kg": solution["stage2_prop_used_kg"],
560         "throttle_min": float(np.min(solution["throttles"])),
561         "throttle_max": float(np.max(solution["throttles"])),
562         "max_collocation_defect": solution["max_collocation_defect"],
563         "iterations": solution["iterations"],
564     }
565     for solution in candidates
566 ]
567 return best
568
569
570 def verify_q4(solution, steps=10_000):
571     coast_time, burn_time = solution["coast_time_s"], solution["burn_time_s"]
572     _, coast = simulate_coast(STAGE1_SEPARATED, coast_time, 2_000)
573     theta = lambda time: interpolate_control(solution["angles_rad"], time /
574 ↪ burn_time)
575     throttle = lambda time: interpolate_control(solution["throttles"], time /
576 ↪ burn_time)
577     times, states = simulate_stage2(coast[-1], burn_time, theta, throttle, steps)
578     residual = terminal_residual(states[-1])
579     return {
580         "times": times,
581         "states": states,
582         "terminal": terminal_metrics(states[-1]),
583         "scaled_terminal_residual": residual,
584         "terminal_residual_norm": float(np.linalg.norm(residual)),
585     }
586
587 def build_full_trajectory(coast_time, burn_time, theta, throttle=None, stage2_steps
588 ↪ =2_000):
589     coast_times, coast = simulate_coast(STAGE1_SEPARATED, coast_time, 500)
590     burn_times, burn = simulate_stage2(coast[-1], burn_time, theta, throttle,
591 ↪ stage2_steps)
592     return combine_segments([(STAGE1_TIME, STAGE1_STATE), (coast_times, coast), (
593 ↪ burn_times, burn)])
594
595

```

```

592 def trajectory_frame(times, states):
593     return pd.DataFrame(
594         {
595             "time_s": times,
596             "height_km": (states[:, 0] - C.re) / 1_000.0,
597             "inertial_speed_m_s": np.hypot(states[:, 2], states[:, 3]),
598             "ground_relative_speed_m_s": np.hypot(states[:, 2], states[:, 3] - C.
↪ omega * states[:, 0]),
599             "flight_path_deg": np.degrees(np.arctan2(states[:, 2], states[:, 3])),
600             "mass_kg": states[:, 4],
601             "radial_speed_m_s": states[:, 2],
602             "tangential_speed_m_s": states[:, 3],
603         }
604     )
605
606
607 def rk4_convergence_table():
608     rows = []
609     for step in [1.0, 0.5, 0.25]:
610         terminal = q1_baseline(step)["terminal"]
611         rows.append(
612             {
613                 "step_s": step,
614                 "final_height_km": terminal["height_km"],
615                 "final_speed_m_s": terminal["inertial_speed_m_s"],
616                 "final_flight_path_deg": terminal["flight_path_deg"],
617                 "final_mass_kg": terminal["mass_kg"],
618             }
619         )
620     return pd.DataFrame(rows)
621
622
623 def loss_budget(times, states, theta_function, throttle_function=None):
624     throttle_function = throttle_function or (lambda _time: 1.0)
625     theta = np.array([theta_function(time) for time in times])
626     throttle = np.array([throttle_function(time) for time in times])
627     gamma = np.arctan2(states[:, 2], states[:, 3])
628     thrust_acceleration = throttle * T2_MAX / states[:, 4]
629     ideal = float(np.trapz(thrust_acceleration, times))
630     steering = float(np.trapz(thrust_acceleration * (1 - np.cos(theta - gamma)),
↪ times))
631     gravity = float(np.trapz(C.mu / states[:, 0] ** 2 * np.sin(gamma), times))
632     speed_gain = float(
633         math.hypot(states[-1, 2], states[-1, 3]) - math.hypot(states[0, 2], states
↪ [0, 3])
634     )
635     return {
636         "ideal_delta_v_m_s": ideal,

```

```

637     "actual_speed_gain_m_s": speed_gain,
638     "gravity_loss_m_s": gravity,
639     "steering_loss_m_s": steering,
640     "balance_error_m_s": ideal - speed_gain - gravity - steering,
641 }
642
643
644 def plot_results(q1, trajectories, angle_series, q4_tau, q4_throttle):
645     frame = trajectory_frame(q1["time_s"], q1["state"])
646     fig, axes = plt.subplots(2, 2, figsize=(11, 7.5), dpi=180)
647     columns = ["height_km", "inertial_speed_m_s", "flight_path_deg", "mass_kg"]
648     labels = ["height / km", "inertial speed / m s-1", "flight-path angle /  
↪ deg", "mass / kg"]
649     for axis, column, label in zip(axes.ravel(), columns, labels):
650         axis.plot(frame["time_s"], frame[column])
651         axis.set(xlabel="time / s", ylabel=label)
652         axis.grid(True, alpha=0.3)
653     fig.suptitle("Q1 baseline trajectory: fixed-step RK4")
654     fig.tight_layout()
655     fig.savefig(FIG_DIR / "q1_baseline_history.png", bbox_inches="tight")
656     plt.close(fig)
657
658     fig, axes = plt.subplots(3, 1, figsize=(11, 9), dpi=180)
659     for label, (times, states) in trajectories.items():
660         data = trajectory_frame(times, states)
661         axes[0].plot(data["time_s"], data["height_km"], label=label)
662         axes[1].plot(data["time_s"], data["inertial_speed_m_s"], label=label)
663         axes[2].plot(data["time_s"], data["flight_path_deg"], label=label)
664     axes[0].axhline(400.0, color="k", lw=0.8, ls="--")
665     axes[1].axhline(V_CIRC, color="k", lw=0.8, ls="--")
666     axes[2].axhline(0.0, color="k", lw=0.8, ls="--")
667     for axis in axes:
668         axis.grid(True, alpha=0.3)
669         axis.legend(fontsize=8)
670     axes[0].set_ylabel("height / km")
671     axes[1].set_ylabel("speed / m s-1")
672     axes[2].set(ylabel="flight-path angle / deg", xlabel="time / s")
673     fig.tight_layout()
674     fig.savefig(FIG_DIR / "trajectory_comparison.png", bbox_inches="tight")
675     plt.close(fig)
676
677     fig, axes = plt.subplots(2, 1, figsize=(10.5, 7), dpi=180)
678     for label, tau, angle in angle_series:
679         axes[0].plot(tau, angle, label=label)
680     axes[0].set_ylabel("pitch angle / deg")
681     axes[0].grid(True, alpha=0.3)
682     axes[0].legend()
683     axes[1].plot(q4_tau, q4_throttle)

```

```

684     axes[1].set(xlabel="normalized second-stage burn time", ylabel="throttle ratio"
↪ , ylim=(0.55, 1.05))
685     axes[1].grid(True, alpha=0.3)
686     fig.tight_layout()
687     fig.savefig(FIG_DIR / "optimized_controls.png", bbox_inches="tight")
688     plt.close(fig)
689
690
691 def json_safe(value):
692     if isinstance(value, dict):
693         return {
694             key: json_safe(item)
695             for key, item in value.items()
696             if key not in {"states_scaled", "angles_rad", "times", "states", "
↪ time_s", "state"}
697         }
698     if isinstance(value, np.ndarray):
699         return value.tolist()
700     if isinstance(value, (np.floating, np.integer)):
701         return value.item()
702     return value
703
704
705 def main():
706     OUT_DIR.mkdir(parents=True, exist_ok=True)
707     FIG_DIR.mkdir(parents=True, exist_ok=True)
708
709     q1, q2 = q1_baseline(0.25), solve_q2()
710     mesh_solutions, previous = [], None
711     for nodes in [11, 21, 31]:
712         solution = solve_q3_collocation(q2, nodes, previous)
713         solution["verification"] = verify_q3(solution, 4_000 if nodes < 31 else 10
↪ _000)
714         mesh_solutions.append(solution)
715         previous = solution
716     q3 = mesh_solutions[-1]
717     q4 = solve_q4_collocation(q3)
718     q4["verification"] = verify_q4(q4, 10_000)
719
720     q2_theta = lambda time: q2["theta0_rad"] + math.radians(q2["rate_deg_s"]) *
↪ time
721     q3_theta = lambda time: interpolate_control(q3["angles_rad"], time / q3["
↪ burn_time_s"])
722     q4_theta = lambda time: interpolate_control(q4["angles_rad"], time / q4["
↪ burn_time_s"])
723     q4_throttle = lambda time: interpolate_control(q4["throttles"], time / q4["
↪ burn_time_s"])
724     trajectories = {

```

```

725     "Q1": (q1["time_s"], q1["state"]),
726     "Q2": build_full_trajectory(q2["coast_time_s"], q2["burn_time_s"], q2_theta
↪ ),
727     "Q3": build_full_trajectory(q3["coast_time_s"], q3["burn_time_s"], q3_theta
↪ ),
728     "Q4": build_full_trajectory(q4["coast_time_s"], q4["burn_time_s"], q4_theta
↪ , q4_throttle),
729 }
730
731 summary_rows = [
732     {"case": "Q1", "coast_time_s": 60.0, "second_stage_burn_s": TB2_MAX, "
↪ stage2_prop_used_kg": MP2, **q1["terminal"]},
733     {"case": "Q2", "coast_time_s": q2["coast_time_s"], "second_stage_burn_s":
↪ q2["burn_time_s"], "stage2_prop_used_kg": q2["stage2_prop_used_kg"], **q2["
↪ terminal"]},
734     {"case": "Q3", "coast_time_s": q3["coast_time_s"], "second_stage_burn_s":
↪ q3["burn_time_s"], "stage2_prop_used_kg": q3["stage2_prop_used_kg"], **q3["
↪ verification"]["terminal"]},
735     {"case": "Q4", "coast_time_s": q4["coast_time_s"], "second_stage_burn_s":
↪ q4["burn_time_s"], "stage2_prop_used_kg": q4["stage2_prop_used_kg"], **q4["
↪ verification"]["terminal"]},
736 ]
737 pd.DataFrame(summary_rows).to_csv(OUT_DIR / "summary.csv", index=False)
738 for label, (times, states) in trajectories.items():
739     trajectory_frame(times, states).to_csv(OUT_DIR / f"{label.lower()}
↪ _trajectory.csv", index=False)
740
741 phase_rows = [
742     {"case": "COMMON", "phase": "lift_off", "time_s": 0.0, **terminal_metrics(
↪ STAGE1_STATE[0])},
743     {"case": "COMMON", "phase": "stage1_burnout_before_separation", "time_s":
↪ TB1, **terminal_metrics(STAGE1_STATE[-1])},
744     {"case": "COMMON", "phase": "stage1_separation_after_jettison", "time_s":
↪ TB1, **terminal_metrics(STAGE1_SEPARATED)},
745 ]
746 for label, coast_time, burn_time, theta, throttle in [
747     ("Q2", q2["coast_time_s"], q2["burn_time_s"], q2_theta, None),
748     ("Q3", q3["coast_time_s"], q3["burn_time_s"], q3_theta, None),
749     ("Q4", q4["coast_time_s"], q4["burn_time_s"], q4_theta, q4_throttle),
750 ]:
751     _, coast = simulate_coast(STAGE1_SEPARATED, coast_time, 500)
752     _, burn = simulate_stage2(coast[-1], burn_time, theta, throttle, 3_000)
753     phase_rows.append({"case": label, "phase": "second_stage_ignition", "time_s
↪ ": TB1 + coast_time, **terminal_metrics(coast[-1])})
754     phase_rows.append({"case": label, "phase": "terminal_shutdown", "time_s":
↪ TB1 + coast_time + burn_time, **terminal_metrics(burn[-1])})
755 pd.DataFrame(phase_rows).to_csv(OUT_DIR / "intermediate_phase_states.csv",
↪ index=False)

```

```

756 pd.DataFrame(q2["iterations"]).to_csv(OUT_DIR / "q2_newton_iterations.csv",
757 ↪ index=False)
758 pd.DataFrame(
759     [
760         {
761             "nodes": solution["nodes"],
762             "coast_time_s": solution["coast_time_s"],
763             "burn_time_s": solution["burn_time_s"],
764             "propellant_kg": solution["stage2_prop_used_kg"],
765             "max_collocation_defect": solution["max_collocation_defect"],
766             "independent_terminal_residual_norm": solution["verification"]["
767 ↪ terminal_residual_norm"],
768             "optimizer_iterations": solution["iterations"],
769         }
770     ]
771 ).to_csv(OUT_DIR / "q3_mesh_convergence.csv", index=False)
772
773 q3_tau, q4_tau = np.linspace(0, 1, q3["nodes"]), np.linspace(0, 1, q4["nodes"])
774 pd.DataFrame(
775     {
776         "node": np.arange(q3["nodes"]),
777         "tau": q3_tau,
778         "radius_m": q3["states_scaled"][:, 0] * R_SCALE,
779         "radial_speed_m_s": q3["states_scaled"][:, 1] * V_SCALE,
780         "tangential_speed_m_s": q3["states_scaled"][:, 2] * V_SCALE,
781         "pitch_deg": np.degrees(q3["angles_rad"]),
782     }
783 ).to_csv(OUT_DIR / "q3_collocation_nodes.csv", index=False)
784 pd.DataFrame(
785     {
786         "node": np.arange(q4["nodes"]),
787         "tau": q4_tau,
788         "radius_m": q4["states_scaled"][:, 0] * R_SCALE,
789         "radial_speed_m_s": q4["states_scaled"][:, 1] * V_SCALE,
790         "tangential_speed_m_s": q4["states_scaled"][:, 2] * V_SCALE,
791         "mass_kg": q4["states_scaled"][:, 3] * M_SCALE,
792         "pitch_deg": np.degrees(q4["angles_rad"]),
793         "throttle": q4["throttles"],
794     }
795 ).to_csv(OUT_DIR / "q4_collocation_nodes.csv", index=False)
796 pd.DataFrame(q4["constant_throttle_audit"]).to_csv(
797     OUT_DIR / "q4_constant_throttle_sensitivity.csv", index=False
798 )
799 pd.DataFrame(q4["multistart_audit"]).to_csv(OUT_DIR / "q4_multistart_audit.csv"
800 ↪ , index=False)
801 rk4_convergence_table().to_csv(OUT_DIR / "rk4_step_convergence.csv", index=

```

```

↪ False)

801
802 loss_rows = []
803 for label, coast_time, burn_time, theta, throttle in [
804     ("Q2", q2["coast_time_s"], q2["burn_time_s"], q2_theta, None),
805     ("Q3", q3["coast_time_s"], q3["burn_time_s"], q3_theta, None),
806     ("Q4", q4["coast_time_s"], q4["burn_time_s"], q4_theta, q4_throttle),
807 ]:
808     _, coast = simulate_coast(STAGE1_SEPARATED, coast_time, 1_000)
809     burn_times, burn = simulate_stage2(coast[-1], burn_time, theta, throttle, 6
↪ _000)
810     loss_rows.append({"case": label, **loss_budget(burn_times, burn, theta,
↪ throttle)})
811 pd.DataFrame(loss_rows).to_csv(OUT_DIR / "second_stage_loss_budget.csv", index=
↪ False)
812
813 angle_series = [
814     ("Q2 constant rate", np.linspace(0, 1, 200), np.degrees([q2_theta(t) for t
↪ in np.linspace(0, q2["burn_time_s"], 200)])),
815     ("Q3 collocation", q3_tau, np.degrees(q3["angles_rad"])),
816     ("Q4 collocation", q4_tau, np.degrees(q4["angles_rad"])),
817 ]
818 plot_results(q1, trajectories, angle_series, q4_tau, q4["throttles"])
819
820 payload = {
821     "constants": {**asdict(C), "target_radius_m": R_TARGET, "circular_speed_m_s
↪ ": V_CIRC, "tb1_s": TB1, "tb2_max_s": TB2_MAX},
822     "methods": {
823         "forward_integration": "fixed-step classical RK4",
824         "q2": "Newton predictor-corrector with a finite-difference sensitivity
↪ matrix and line search",
825         "q3_q4": "Hermite-Simpson direct collocation followed by independent
↪ RK4 verification",
826     },
827     "Q1": json_safe(q1),
828     "Q2": json_safe(q2),
829     "Q3": json_safe(q3),
830     "Q4": json_safe(q4),
831 }
832 (OUT_DIR / "results.json").write_text(json.dumps(payload, ensure_ascii=False,
↪ indent=2), encoding="utf-8")
833 print(pd.DataFrame(summary_rows).to_string(index=False))
834 print("\nQ3 mesh convergence")
835 print(pd.read_csv(OUT_DIR / "q3_mesh_convergence.csv").to_string(index=False))
836
837
838 if __name__ == "__main__":
839     main()

```
